BEYOND FAILED TRAJECTORIES: DIAGNOSING AND REPAIRING LLM AGENT HARNESSES ACROSS THREE OBSERVABILITY LEVELS

Anonymous authors
Paper under double-blind review

ABSTRACT

Failure-driven debugging of LLM-agent harnesses implicitly assumes that the execution selected for diagnosis already contains evidence of the defect. We show that successful executions violate this assumption in two distinct ways. In L2, the task succeeds but the trajectory contains abnormal evidence; in L3, the ordinary execution remains healthy until another reachable condition exposes the defect. Across 9,058 harness-related issues and merged pull requests from 16 open-source agent projects, 25.1% of records with defined levels are L2 or L3, with high agreement in independent audits by two authors. A controlled study of 36 reconstructed historical defects further shows that passive source inspection can raise suspicion about latent defects without confirming their manifestation, and that source localization remains a separate bottleneck once an exposing execution is available. We introduce *Proactive Invariant Harness Tomography* (PROACTIVE IHT), which audits all trajectories, routes successful runs with invariant violations to diagnosis, selectively probes healthy candidates for missing evidence, and verifies candidate mechanisms before scoped repair. Across four public benchmarks and four model backbones, Proactive IHT improves over HarnessFix in all 16 matched settings by 2.0–5.0 percentage points in task completion rate (TCR), with an average gain of 3.6 points. These results show that harness repair benefits from treating evidence acquisition as part of the debugging problem rather than using task failure as its only entry point.

Code | Website

1 INTRODUCTION

Large language model (LLM) agents depend on a *harness*: runtime code that builds model context, exposes tools, stores state, orchestrates execution, and decides when a task is complete. Changes to this code can alter agent behavior even when the base model is fixed (Yao et al., 2023; Schick et al., 2023; Yang et al., 2024; Wang et al., 2025). Recent work has consequently begun to optimize or repair the harness itself (Lou et al., 2026; Lee et al., 2026; Pan et al., 2026; Lin et al., 2026; Chen et al., 2026b).

Most debugging begins from a failed trajectory. This makes an implicit assumption: the execution selected for diagnosis already contains evidence of the defect. Successful executions break this assumption in two distinct ways. A run can contain an abnormal signal—for example, a swallowed tool error, inconsistent state, or bypassed verification—while still completing the task, or it can appear healthy because the defective branch or boundary condition was never exercised. The former can be investigated from the existing trajectory; the latter requires an informative new execution.

We call this distinction *harness defect observability*. Figure 1a separates three execution-level regimes. Level 1 (L1) is outcome-visible: a defect causes task failure. Level 2 (L2) is outcome-silent but trajectory-visible: the task succeeds while the trajectory contains abnormal evidence. Level 3 (L3) is trajectory-silent under the ordinary execution: the task succeeds and its observed trajectory is healthy, but another reachable condition exposes the defect. The levels characterize what an execution reveals, not defect severity. The resulting question is: *what evidence must a harness debugger acquire before diagnosis and repair become justified?*

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

Our empirical study establishes that this boundary matters in practice. From 15,293 unique public issues and merged pull requests across 16 open-source agent projects, a two-stage DeepSeek-V4-Flash pipeline retains 9,058 harness-related records and labels 9,045 of them. L1, L2, and L3 account for 74.9%, 8.2%, and 16.9%, respectively; hence 25.1% preserve task success. Under conservative joint confirmation, two authors audit 100 records per level and confirm 100/100 L1, 95/100 L2, and 96/100 L3 labels; a second audit confirms 91/100 sampled unconditional-L3 records. Thus, restricting diagnosis to failed runs systematically omits the two successful regimes observed in the corpus.

This evidence view yields *Proactive Invariant Harness Tomography* (PROACTIVE IHT). Every trajectory is audited against runtime invariants. Failed runs provide L1 evidence; successful runs with relevant violations provide L2 evidence; successful runs without relevant violations become healthy/L3 candidates for bounded, targeted activation. Once behavioral evidence is available, all routes use the same source-based diagnosis, mechanism verification, scoped repair, and regression validation. Section 4 formalizes this boundary using standard decision-theoretic relations: richer observation can reveal L2 evidence that is already present, whereas an ordinary L3 execution remains non-separating and requires a change in execution condition. When a defect manifests with probability p under passive execution, the expected waiting time for the first witness is $1/p$.

We evaluate evidence acquisition and repair separately. On 36 reconstructed historical defects, passive methods show complementary limitations, while source localization remains a distinct bottleneck even after an exposing execution is supplied. In closed-loop evaluation, PROACTIVE IHT improves over HarnessFix in all 16 matched benchmark and backbone settings by 2.0–5.0 TCR points (3.6 points on average). On AppWorld, auditing all 90 training trajectories routes 22 successful runs to L2-like invariant evidence, and activation produces relevant new evidence for 38/45 healthy/L3 candidates. With GPT-5 mini, the average gain over HarnessFix is 2.3 TCR points for a 10.7% increase in offline repair tokens. The ablation separates these effects: L1-only routing retains 74% of the full gain over HF, while routing successful runs broadens the available evidence and the shared diagnosis and verification stages account for larger changes in TCR.

Our contributions are:

- an execution-level account of harness defect observability showing that 25.1% of 9,045 labeled development records preserve task success, while differing in whether the trajectory itself exposes defect evidence;
- a controlled study of 36 historical defects that separates limits in evidence acquisition from limits in source localization; and
- PROACTIVE IHT, which implements evidence acquisition tailored to the three observability levels before shared diagnosis, verification, and repair, and improves over HarnessFix across 16 matched benchmark and backbone settings.

2 RELATED WORK

Harness optimization and trajectory-based repair. Tool use and long-horizon interaction make runtime design part of an agent's capability (Yao et al., 2023; Schick et al., 2023; Yang et al., 2024; Wang et al., 2025). Tool-oriented and multi-agent systems further expose how API interfaces, orchestration, and persistent interaction shape agent behavior (Patil et al., 2023; Qin et al., 2023; Li et al., 2023b; Wu et al., 2023; Li et al., 2023a; Hong et al., 2024; Wang et al., 2023). Recent systems synthesize, search over, or evolve this layer from environment feedback or logged experience (Lou et al., 2026; Lee et al., 2026; Pan et al., 2026; Lin et al., 2026); related agent-level methods optimize broader adaptation surfaces through reflection and prompt evolution (Shinn et al., 2023; Agrawal et al., 2026; Zhang et al., 2026b). Open-ended self-improvement extends this surface further (Zhang et al., 2026a). A complementary line diagnoses failures from execution trajectories using structured traces, replay, injected faults, or interventions (Zhang et al., 2025b; Barke et al., 2026; Ou et al., 2025; Liu et al., 2026; Zhang et al., 2025a; Ge et al., 2025; Ma et al., 2025). HarnessFix is closest to our repair setting: it starts from failed trajectories, builds a harness-aware representation, diagnoses flaws, and generates scoped patches (Chen et al., 2026b).

These approaches address diagnosis or optimization once useful execution evidence is available. Our focus is the preceding question: whether the current execution provides enough behavioral evidence to justify diagnosis at all. HarnessFix uses failure as its entry point; intervention-based debugging

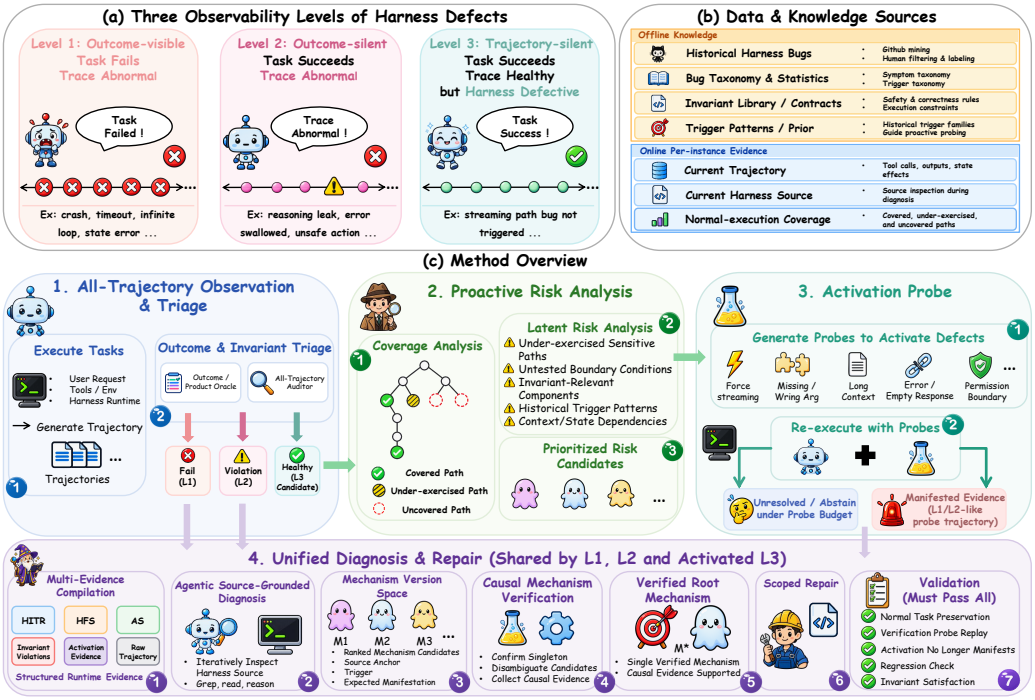


Figure 1: **Overview of Proactive IHT.** (a) The three observability levels differ by what an ordinary execution reveals: L1 is outcome-visible, L2 preserves task success but exposes abnormal trajectory evidence, and L3 is trajectory-silent until another reachable condition activates the defect. (b) Historical data provide trigger and priors over defect locations while evaluator-only artifacts remain hidden. (c) The auditor routes L1 and L2 evidence directly to diagnosis; successful runs with no active invariant violations become healthy/L3 candidates and receive targeted activation. Once evidence exists, all routes share source-grounded diagnosis, mechanism verification, repair, and validation.

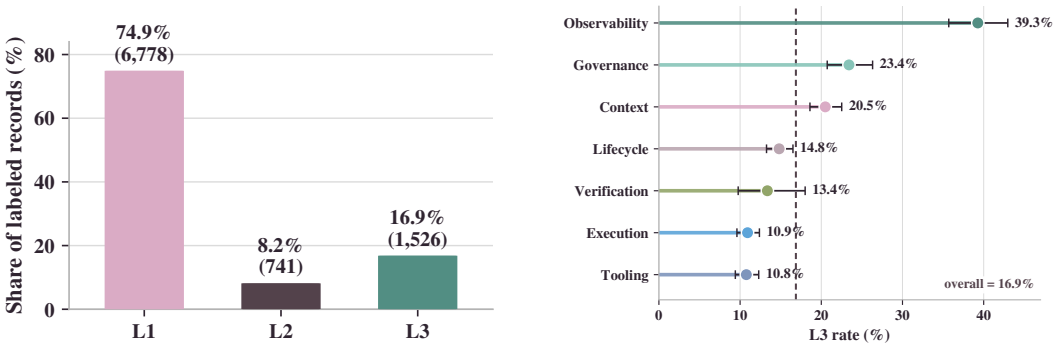
such as DoVer tests causal hypotheses about observed failures; and coverage-guided approaches such as FLARE search for executions that expose failures (Ma et al., 2025; Hui et al., 2026). In contrast, we first distinguish successful executions whose trajectories already contain abnormal evidence from successful executions that remain behaviorally uninformative, and use that distinction to decide when an additional execution is necessary. This use of observability also differs from observability-driven harness evolution (Lin et al., 2026): here observability characterizes what a particular execution can establish about a defect.

Empirical studies and active fault exposure. Empirical studies of model–harness bugs and agent repository issues commonly organize cases by cause, symptom, or repair category (Chen et al., 2026a; Rahardja et al., 2025; Majgaonkar et al., 2025; Mulian et al., 2026). Broader agent benchmarks likewise emphasize interactive trajectories across software, web, desktop, and safety settings (Liu et al., 2024; Xie et al., 2024; Rawles et al., 2025; Li et al., 2026). We instead organize cases by the evidence available from an execution: task failure, successful execution with abnormal trajectory evidence, or successful execution without a behavioral witness. This view connects to software testing, where property-based testing, directed fuzzing, and coverage-guided methods construct executions to expose faults (Claessen & Hughes, 2000; Yang et al., 2011; Böhme et al., 2017; Godefroid et al., 2017; Xie et al., 2018); LLM-based fuzzers extend this idea to learned inputs and agent systems (Deng et al., 2023; Xia et al., 2024; Hui et al., 2026). Related LLM-based program-repair systems use autonomous diagnosis and patch generation over source code (Bouzenia et al., 2024; Zhang et al., 2024). PROACTIVE IHT uses active execution more narrowly: it probes a bounded set of healthy candidates to obtain missing behavioral evidence, and later uses mechanism-specific interventions to support or reject candidate diagnoses before repair.

3 Empirical Study: Three Levels of Harness Defect Observability

How often does a real harness report provide the execution evidence assumed by trajectory-based debugging? We collect 15,293 unique public issues and merged pull requests from 16 open-source agent projects. URL deduplication and feature-oriented filtering leave 12,952 candidates. A two-stage DeepSeek-V4-Flash pipeline identifies harness-related records and annotates trigger, manifestation, task impact, trace evidence, and observability from the issue/PR text and available fix diff. It retains 9,058 harness-related records, of which 9,045 receive a defined L1/L2/L3 label. Appendix E documents prompts, fields, filtering, and audits; the corpus labels are development-record evidence rather than replays of every issue.

Labels and audit. L1 denotes a bug that causes task failure; L2 preserves the task outcome but leaves bug-relevant abnormal trajectory evidence; L3 leaves the ordinary run healthy because the affected behavior or trigger is absent. Two authors independently inspect 100 LLM-assigned records from each level against the underlying issue/PR evidence, counting a confirmation only when both agree. They confirm 100/100 L1, 95/100 L2, and 96/100 L3 labels (291/300 overall). The executable suite in Section 6 uses a stricter runtime criterion: an L2 case must preserve product success while violating an obligation, and an L3 case must pass a benign ordinary execution with a healthy trace while another reachable condition exposes the defect.



(a) Overall distribution

(b) L3 rate by harness layer

Figure 2: **Empirical observability landscape.** (a) Labels for the 9,045 records with a defined level. (b) L3 stratification by harness layer; remaining stratifications and Wilson 95% intervals are in Appendix E.

Successful reports are not rare, and are not evidence-equivalent. Of the labeled records, 6,778 (74.9%) are L1, 741 (8.2%) L2, and 1,526 (16.9%; Wilson 95% CI: 16.1–17.7%) L3. Thus 25.1% preserve task success. A failure-only trigger has no positive outcome signal for these reports, but L2 and L3 require different responses: successful L2 runs remain auditable, while an ordinary L3 run is non-separating.

The regimes also cut across mechanism and impact. L2 is concentrated where a harness may violate an obligation without immediately changing task score: its rate reaches 20.4% in observability and 12.4% in governance, versus 8.2% corpus-wide. L3 occurs in all 16 projects, with the highest layer-level rate in observability (39.2%), followed by governance (23.4%) and context (20.4%) (Figure 2b). It is not synonymous with low impact: resource waste, security leak, state corruption, and silent no-op have L3 rates of 59.3%, 45.3%, 38.7%, and 35.3%, respectively. Nor does a report usually provide a replay: only 339/1,526 L3 records (22.2%) contain explicit reproduction steps or a linked test/PR reproducer. Detailed impact, trigger, project, and replay analyses are in Appendix E.

The study motivates a level-aware policy: failed outcomes route L1; all-trajectory checks can surface L2; and L3 calls for a changed execution condition when the ordinary trajectory supplies no witness.

4 FORMALIZING THE EVIDENCE BOUNDARY

We use standard decision-theoretic relations to formalize the evidence boundary suggested by the empirical study. Let H_{clean} and H_{def} denote healthy and defective harnesses. An execution condition x includes the task, model output, runtime state, and configuration; running H produces outcome y and trajectory τ. Outcome-only debugging observes y, whereas a trajectory-aware debugger observes $O = \psi(y, \tau)$. L1 changes the outcome. L2 can preserve the outcome while changing O. In an ordinary L3 execution, the relevant event does not occur, so even a full retained trajectory can remain non-separating.

For an execution policy P, let P^P_{clean} and P^P_{def} be the induced distributions of O, and define

$$\Omega_P(H_{\text{clean}}, H_{\text{def}}) = \text{TV}(P^P_{\text{clean}}, P^P_{\text{def}}). \tag{1}$$

This is a policy-level property, while L1/L2/L3 label individual executions.

Proposition 1 (Behavioral identifiability)**.** *With equal priors, the minimum error of a detector using only O is*

$$R^*_P = \frac{1 - \Omega_P(H_{\text{clean}}, H_{\text{def}})}{2}. \tag{2}$$

Thus $\Omega_P = 0$ makes the two harnesses indistinguishable from the observed execution.

Richer observation can expose an L2 abnormality that is already present; it cannot create evidence for an L3 event that never occurred. Source code can alter which mechanism appears plausible, but not establish its manifestation. A second limit is exposure frequency. Let p_d be the probability that a passive execution falls in the distinguishing-condition set for defect d.

Proposition 2 (Cost of passive exposure)**.** *For independent executions with stable $p_d \in (0, 1)$, if T is the first manifestation time,*

$$\mathbb{E}[T] = \frac{1}{p_d}, \qquad \Pr[T > n] = (1 - p_d)^n. \tag{3}$$

Consequently, observing a witness with probability at least $1 - \delta$ requires

$$n \geq \left\lceil \frac{\log \delta}{\log(1 - p_d)} \right\rceil \approx \frac{\log(1/\delta)}{p_d} \quad \textit{for small } p_d. \tag{4}$$

A targeted policy changes the execution distribution and is useful only insofar as it increases distinguishing manifestations for a supported candidate mechanism. PROACTIVE IHT therefore audits all successful runs for L2 evidence and reserves bounded activation for healthy/L3 candidates with no active invariant violations. Proofs and scope assumptions are in Appendix I.

5 PROACTIVE INVARIANT HARNESS TOMOGRAPHY

PROACTIVE IHT separates two questions: what does the current execution establish, and which harness mechanism explains that evidence? Figure 1c summarizes the resulting closed loop. Every run is audited; the evidence route determines whether the method diagnoses immediately or must acquire an additional execution. A route is not a final defect label: a failure may be an agent-capability error, an invariant violation still needs attribution, and an invariant-clean success becomes L3 only after a reachable activation exposes a defect.

All-trajectory routing. For evaluator outcome $y \in \{0, 1\}$ and invariant violations $V(\tau)$, routing is

$$\text{route}(\tau) = \begin{cases} \text{failure-visible candidate}, & y = 0, \\ \text{trajectory-abnormal candidate}, & y = 1 \wedge V(\tau) \neq \emptyset, \\ \text{healthy candidate}, & y = 1 \wedge V(\tau) = \emptyset. \end{cases} \tag{5}$$

Unknown outcomes and checker failures are excluded from the healthy branch. The invariant registry contains cross-harness and repository-specific obligations over tooling, context, lifecycle, observability, verification, governance, and execution. Examples include preserving tool errors, marking

context truncation, respecting approval/retry boundaries, and keeping reported state consistent with actual effects. Appendix B gives the registry and calibration.

Each evidence-bearing execution is compiled into a common bundle: a HarnessFix-style Harness-aware Trace Intermediate Representation (HTIR) (Chen et al., 2026b), the raw trace, invariant findings, and compact fault-signal and adaptation signatures. The former records outcome, surfaced or swallowed errors, retries, termination, truncation, and state/effect signals; the latter records agent reactions such as retry, fallback, or tool switching. The bundle is evidence for diagnosis, not a root-cause claim.

Targeted activation for healthy/L3 candidates. For an invariant-clean success, PROACTIVE IHT uses ordinary-run coverage only to prioritize unexercised or boundary-sensitive behavior. A risk analyzer ranks bounded targets using coverage, invariant relevance, current source context, and trigger patterns mined from disjoint historical bugs. Each target specifies a source anchor, proposed reachable condition, and expected manifestation; evaluator-only issue text, fixed source, developer diffs, gold tests, and known reproducers remain hidden.

The executor applies the highest-ranked feasible interventions, such as a configuration boundary, controlled model response, task context, permission/retry boundary, or supported cross-session state. For each RQ4 benchmark, it re-runs the benchmark's native task runner and evaluator in an isolated copy of the current harness; the registered execution condition is the only intentional difference. An application must be verified; unsupported operations and timeouts are recorded rather than treated as evidence. A probe is relevant only if it changes success to failure, produces a target invariant violation, or realizes the expected symptom class. Otherwise, or when the budget ends, the method abstains and retains an unresolved healthy candidate. Operational contracts are in Appendix D.

Source-grounded diagnosis and causal verification. Once behavioral evidence exists, the diagnoser receives

$$E = \{\text{HTIR}, \text{HFS}, \text{AS}, V(\tau), E_{\text{act}}, \tau\} \tag{6}$$

and a sanitized source snapshot. Within a bounded interaction budget, it returns ranked mechanism candidates with source anchors, trigger condition, expected manifestation, supporting evidence, and failure nature. This separates harness defects from agent-capability failures. RQ3 isolates this step after a witness is supplied; its error analysis motivates tracing control and data relations from a behavior site to the component that *defines* that behavior.

A candidate is repaired only after a confirmation or disambiguation intervention. Conceptually, a probe trades information against cost,

$$p^* = \arg\max_{p \in \mathcal{P}} I(M; Z_p \mid E) - \lambda C(p), \tag{7}$$

while the implementation uses candidate-specific evidence, expected manifestations, source locality, and scope as a practical surrogate. Support requires a candidate-specific change: an L2 anomaly disappears, or an activated L3 manifestation disappears while the activation condition is held fixed. Contradictory or uninformative probes leave the candidate unverified; exhausted budgets skip repair.

Scoped repair and validation. Verified records become flaw-level specifications binding source anchors and causal evidence to an allowed edit scope, required post-repair behavior, and rollback conditions. Static guards reject edits to evaluators or probe machinery, out-of-scope changes, syntax failures, and task-answer hard-coding. Validation replays the verification evidence (and the original L3 activation when applicable), requires target L2 violations to disappear, preserves benign behavior and active invariants, and checks the broader regression suite. Appendices C and D provide detailed contracts.

6 EXPERIMENTAL DESIGN

The evaluation follows the evidence boundary. RQ1 measures whether L1/L2/L3 occur in real development records. RQ2 asks what passive diagnosis can establish when intervention is disabled. RQ3 supplies a level-specific witness and measures source localization separately. RQ4 evaluates the full repair loop, keeping held-out task utility distinct from successful-run evidence coverage.

RQ1: empirical prevalence. Section 3 reports the 9,058-record study, stratifications, and independent audits. Mining, annotation, model-version, and audit details are in Appendix E.

RQ2–RQ3: controlled historical defects. OBSERVABILITYBENCH reconstructs 36 verified defects from CAMEL, CrewAI, LangChain, and OpenAI Agents (12 L1, 8 L2, and 16 L3). Every case has paired buggy/fixed revisions, an executable driver and verifier, a benign control, and a level-specific defect-exposing execution. For L3, the benign run is healthy on the buggy revision while a separate reachable probe exposes the defect and remains safe on the fixed revision. Admission rules and artifacts appear in Appendix F.

In RQ2, methods may not choose or execute an intervention. Raw LLM, HarnessFix (HF) (Chen et al., 2026b), an invariant Auditor, and Passive IHT are evaluated on trigger-pair discrimination, benign specificity, and single-execution regime classification. Passive IHT adds HTIR, signatures, and source inspection but disables activation and mechanism verification. The L3 classification input is deliberately its healthy benign run, not the controlled probe: source suspicion does not count as manifested runtime evidence. All model-based calls use the same DeepSeek-v4.1-Flash endpoint with reasoning disabled; prompts, decoding, intervals, and confusion matrices are in Appendix G.

RQ3 removes observability as a confound: each method receives the level and an execution exposing the historical defect (failure for L1, anomalous success for L2, and probe witness for L3), but not the fixed revision, gold source, issue/fix metadata, or verifier-only state. Under a matched source-inspection budget, methods return ranked source anchors. Candidate-anchor recall at K (CR@K) is a hit when a top-K anchor matches a Python file changed by the historical fix inside method-visible harness roots. The frozen IHT configuration is the primary result. Same-suite post-hoc attribution recovery is retained only as Appendix H.1 error analysis, not an independent generalization estimate.

RQ4: matched closed-loop repair. We mirror the HarnessFix train–modify–validate–promote–test protocol on matched benchmark/harness pairs: GAIA/open-deep-research (60/30/60) (Mialon et al., 2024), SWE-Bench Verified/mini-swe-agent (100/50/100) (Jimenez et al., 2024; OpenAI, 2024), AppWorld/official simplified ReAct code agent (90/45/90) (Trivedi et al., 2024), and Terminal-Bench 2.0 Verified/Harbor Terminus-2 (34/17/34) (Merrill et al., 2026). The sampled split sizes and harness pairings follow HarnessFix (Chen et al., 2026b). The four backbones are DeepSeek-v4.1-Flash, MiniMax-M2.5, Qwen3.6-Flash, and HY3. Within every benchmark/backbone row, the initial harness, split, model settings, source visibility, repair budget, patch audit, validation gate, and evaluator are identical for HF and PROACTIVE IHT. Each promoted harness is evaluated in three held-out test runs, and TCR is their arithmetic mean. Exact configurations are in Appendix H.2.

The production cost policy audits all training trajectories: all L1 and L2-like evidence enters diagnosis, while L3 activation is capped at two healthy candidates per optimization run. This treats L3 as an additional controlled execution, not a free extension of trace analysis. The GPT-5-mini table reuses the published baseline rows from HarnessFix and adds a newly evaluated PROACTIVE IHT row on the reported initial harnesses and splits; full provenance and token accounting are in Appendix L.

For evidence characterization rather than repair selection, we also exhaustively route the 90 AppWorld DeepSeek-v4.1-Flash training trajectories: we retain supporting evidence for every L2-like route and activate every healthy/L3 candidate. Auditor flags and activation manifestations are not called verified defects without later mechanism verification. Finally, a Qwen3.6-Flash ablation uses the same four splits, repair budget, validation gate, and three-test evaluation. It removes multi-view diagnosis evidence, routing of successful runs, L3 activation, or causal verification while keeping the specified remaining components fixed. TCR is primary; GPT-5-mini offline optimization/repair tokens exclude benchmark task-run tokens. Full protocol is in the appendices.

7 RESULTS

7.1 PASSIVE EVIDENCE AND SOURCE LOCALIZATION ARE SEPARATE LIMITS (RQ2–RQ3)

Table 1 first isolates what a method can establish without intervention. In the classification task based on a single passive execution, L1 already supplies a failure witness: Raw LLM, Auditor, and Passive IHT classify 11–12 of 12 cases correctly. L2 instead requires looking beyond success. The frozen Auditor and Passive IHT classify only 1/8 L2 cases correctly, whereas Raw LLM classifies

5/8; inspection attributes the former error to conservative short-trace filters that suppress both noise and an anomaly. This is an invariant-calibration limitation, not an activation limitation.

For L3, the input is an intentionally healthy ordinary run. HarnessFix flags 0/16 cases as L3 candidates because it never enters its failed-trajectory path. Passive IHT flags all 16 as candidates for a latent defect by combining source inspection with richer summaries, but it returns *neither* for only 5/36 benign buggy/fixed pairs. Raw LLM and Auditor have the complementary profile: they achieve 35/36 benign specificity but flag only 5/16 and 8/16 L3 cases, respectively. These L3 predictions are hypotheses about latent risk, not behavioral confirmation of a defect. The result therefore supports auditing successful trajectories for L2 evidence and using activation only when the ordinary execution provides no behavioral witness.

RQ3 supplies an exposing execution and the level, then asks whether diagnosis can recover the responsible source mechanism. The raw source-aware LLM reaches CR@3=10/36; HarnessFix reaches 18/36; and frozen IHT reaches 17/36, despite its higher CR@1 (11/36). A witness therefore does not remove the source-attribution bottleneck. The same-suite post-hoc recovery is reported only as Appendix H.1 error analysis: it traces from a behavior site to the component that defines the behavior, rather than claiming an independent estimate.

Table 1: **Passive evidence and post-exposure localization on 36 historical defects.** (a) Classification from a single passive execution and benign specificity. The L3 column records candidate flags from the healthy ordinary run, not behavioral confirmation. (b) Candidate-anchor recall after the exposing execution and level are supplied. Full paired decisions, intervals, prompts, and same-suite error analysis are in Appendices G and H.1.

(a) RQ2: passive evidence

Method	L1	L2	L3 flag	All	Benign
Raw LLM	11/12	**5/8**	5/16	21/36	**35/36**
Auditor	**11/12**	1/8	8/16	20/36	**35/36**
Passive IHT	**11/12**	1/8	**16/16**	**28/36**	5/36
HarnessFix	8/12	0/8	0/16	8/36	1/36

(b) RQ3: CR@K after exposure

Method	CR@1	CR@3	CR@5
Raw LLM	5/36	10/36	12/36
HarnessFix	9/36	**18/36**	**21/36**
Frozen IHT	**11/36**	17/36	18/36

7.2 Broader evidence supports consistent, bounded repair gains (RQ4)

Table 2 is the controlled closed-loop comparison: within each row, HF and Proactive IHT use the same initial harness, split, model settings, source visibility, repair budget, patch audit, validation gate, and evaluator, and each TCR is the mean of three held-out tests. Proactive IHT improves over HF in all 16 matched settings by 2.0–5.0 points (3.6 on average). The gains are consistent across the four benchmarks (mean 3.9 on GAIA, 3.3 on SWE-Bench Verified, 3.6 on AppWorld, and 3.4 on TB2), but modest relative to the shared $H_0 \rightarrow$HF improvement. This is the intended comparison: the methods share an outer repair loop and differ primarily in evidence acquisition, diagnosis evidence, and verification.

All rows use the production evidence budget: all L1 and L2-like training trajectories enter diagnosis, while activation is capped at two healthy/L3 candidates per optimization run. The broader GPT-5-mini comparison is in Appendix L: Proactive IHT averages 49.4 TCR versus 47.1 for HF (+2.3 points), using 49.0M versus 44.3M offline repair tokens (+10.7%).

Evidence coverage explains why TCR alone is incomplete but does not establish prevalence. In the exhaustive AppWorld characterization, 23/90 training trajectories are L1-like, 22/90 successful trajectories are L2-like, and 45/90 are healthy/L3 candidates. Activating all 45 candidates yields relevant manifestations in 38 cases (28 L1-like and 10 L2-like). This is a characterization of the method's prioritized risk region, not a count of verified defects or an unbiased estimate of defect prevalence; full accounting is in Appendix M.

The ablation bounds the causal reading of RQ4. Full IHT gains 4.6 points over HF; L1-only retains 3.4 points, or 74% of that gain. Restoring L2 raises the average by 0.9 points, and the fixed two-candidate L3 branch adds 0.3 points. In contrast, removing multi-view diagnosis evidence or causal verification costs 3.5 and 2.2 points relative to full IHT. Under the deliberately small two-candidate activation budget, routing successful runs primarily expands the evidence available to the repair loop; most

Table 2: **Matched closed-loop comparison across four model backbones.** TCR is the mean of three held-out test runs. Δ is PROACTIVE IHT minus HarnessFix in percentage points.

Benchmark	Backbone	H_0	H_0+HF	H_0+IHT	Δ
GAIA	DeepSeek-v4.1-Flash	65.6	73.9	77.2	+3.3
	MiniMax-M2.5	66.7	76.1	80.6	+4.4
	Qwen3.6-Flash	62.2	72.8	77.8	+5.0
	HY3	72.2	79.4	82.2	+2.8
SWE-Bench Ver.	DeepSeek-v4.1-Flash	66.0	74.0	76.7	+2.7
	MiniMax-M2.5	68.0	75.3	79.0	+3.7
	Qwen3.6-Flash	64.0	73.7	78.3	+4.7
	HY3	69.3	76.0	78.3	+2.3
AppWorld	DeepSeek-v4.1-Flash	69.3	76.7	79.6	+3.0
	MiniMax-M2.5	56.3	66.3	70.4	+4.1
	Qwen3.6-Flash	53.7	64.8	69.6	+4.8
	HY3	78.5	84.8	87.4	+2.6
TB2	DeepSeek-v4.1-Flash	49.0	56.9	59.8	+2.9
	MiniMax-M2.5	42.2	52.9	57.8	+4.9
	Qwen3.6-Flash	51.0	59.8	63.7	+3.9
	HY3	56.9	63.7	65.7	+2.0

Table 3: **Qwen3.6-Flash ablation.** TCR is the mean of three held-out tests; all variants share splits, outer repair budget, and validation gate.

Variant	GAIA	SWE	AppW.	TB2	Avg.
H_0	62.2	64.0	53.7	51.0	57.7
HarnessFix	72.8	73.7	64.8	59.8	67.8
w/o multi-view	73.3	74.3	67.0	60.8	68.9
L1-only	76.1	77.0	68.9	62.7	71.2
w/o L3	77.2	78.0	69.3	63.7	72.1
w/o verification	75.0	76.3	67.8	61.8	70.2
Full IHT	**77.8**	**78.3**	**69.6**	**63.7**	**72.4**

of the TCR gain comes from the shared multi-view diagnosis and verification stages, which also improve repairs for L1 failures.

8 CONCLUSION

Harness debugging requires evidence before it can justify a repair. We distinguish L1 executions that expose a defect through task failure, L2 executions that preserve task success but contain abnormal trajectory evidence, and L3 executions that remain behaviorally uninformative until another reachable condition exposes the defect. In 9,058 harness-related development records, L2 and L3 together account for 25.1% of labeled reports. Task failure is therefore an incomplete trigger for diagnosis: trajectory auditing can surface L2 evidence, whereas an L3 candidate requires an additional informative execution.

PROACTIVE IHT operationalizes this distinction by making evidence acquisition part of the repair loop. It improves over HarnessFix in all 16 matched settings by 2.0–5.0 TCR points (3.6 on average). L1-only routing retains 74% of the full gain, while the successful L2/L3 routes contribute smaller increments under the fixed activation budget. The broader result is that reliable harness repair should distinguish evidence acquisition from source localization and repair, rather than assume that a failed task is the only useful starting point.

REPRODUCIBILITY STATEMENT

The main paper specifies the observability levels, controlled tasks, and matched closed-loop comparison. The appendices provide mining and audit procedures, executable-case construction, prompts,

per-case outputs, configurations, and complete AppWorld routing/activation accounting. Historical fixed revisions and target-issue artifacts remain evaluator-only; provider-returned model identifiers and decoding parameters are logged for final runs. RQ4 rows and ablation variants use three independent held-out test runs.

Ethics Statement

We study public historical or responsibly disclosed software defects in isolated environments with synthetic credentials and controlled services. Release artifacts should omit secrets, internal infrastructure identifiers, and operational details that would enable exploitation of unremediated security defects.

Artificial Intelligence (AI) Use Statement

Generative AI tools were used to aid or polish manuscript writing and to support literature retrieval and discovery. They also assisted in constructing ObservabilityBench by helping generate and refine the executable benchmark artifacts used to reconstruct and evaluate historical harness defects. The main codebase was written by the authors; AI tools were used for code review, debugging, and optimization of implementation logic. All AI-assisted outputs were reviewed and verified by the authors, who remain responsible for the final manuscript, code, benchmark artifacts, references, analyses, and experimental results.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=RQm2KQTM5r.

Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. AgentRx: Diagnosing AI agent failures from execution trajectories, 2026. URL https://arxiv.org/abs/2602.02475.

Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, pp. 2329–2344. ACM, 2017. doi: 10.1145/3133956.3134020. URL https://doi.org/10.1145/3133956.3134020.

Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair, 2024. URL https://arxiv.org/abs/2403.17134.

Jingyi Chen, Songqiang Chen, Hengcheng Zhu, Jialun Cao, Jiasi Shen, and Shing-Chi Cheung. Understanding agent-reactive bugs at the model-harness boundary: An empirical study of LLM agent issue reports, 2026a. URL https://arxiv.org/abs/2607.15684.

Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws, 2026b. URL https://arxiv.org/abs/2606.06324.

Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of haskell programs. *ACM SIGPLAN Notices*, 35(9):268–279, 2000. doi: 10.1145/357766.351266. URL https://doi.org/10.1145/357766.351266.

Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models, 2023. URL https://arxiv.org/abs/2212.14834.

Yu Ge, Linna Xie, Zhong Li, Yu Pei, and Tian Zhang. Who is introducing the failure? automatically attributing failures of multi-agent systems via spectrum analysis, 2025. URL https://arxiv.org/abs/2509.13782.

Patrice Godefroid, Hila Peleg, and Rishabh Singh. Learn&Fuzz: Machine learning for input fuzzing, 2017. URL https://arxiv.org/abs/1701.07232.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VtmBAGCN7o.

Mingxuan Hui, Xinyue Li, Lu Wang, Chengcheng Wan, Yifan Wang, Yimian Wang, Feiyue Song, Beining Shi, Yixi Li, and Yaxiao Li. FLARE: Agentic coverage-guided fuzzing for LLM-based multi-agent systems, 2026. URL https://arxiv.org/abs/2604.05289.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL https://arxiv.org/abs/2603.28052.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society, 2023a. URL https://arxiv.org/abs/2303.17760.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A comprehensive benchmark for tool-augmented LLMs, 2023b. URL https://arxiv.org/abs/2304.08244.

Yu Li, Haoyu Luo, Yuejin Xie, Yuqian Fu, Zhonghao Yang, Shuai Shao, Qihan Ren, Wanying Qu, Yanwei Fu, Yujiu Yang, Jing Shao, Xia Hu, and Dongrui Liu. ATBench: A diverse and realistic agent trajectory benchmark for safety evaluation and diagnosis, 2026. URL https://arxiv.org/abs/2604.02022.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL https://arxiv.org/abs/2604.25850.

Shuyang Liu, Yang Chen, Rahul Krishna, Saurabh Sinha, Jatin Ganhotra, and Reyhaneh Jabbarvand. Process-centric analysis of agentic software systems. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1):1961–1988, 2026. doi: 10.1145/3798271. URL https://doi.org/10.1145/3798271.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=zAdUB0aCTQ.

Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving LLM agents by automatically synthesizing a code harness, 2026. URL https://arxiv.org/abs/2603.03329.

Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. DoVer: Intervention-driven auto debugging for LLM multi-agent systems, 2025. URL https://arxiv.org/abs/2512.06749.

Oorja Majgaonkar, Zhiwei Fei, Xiang Li, Federica Sarro, and He Ye. Understanding code agent behaviour: An empirical study of success and failure trajectories, 2025. URL https://arxiv.org/abs/2511.00197.

Mike A. Merrill et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=fibxvahvs3.

Hadar Mulian, Sergey Zeltyn, Ido Levy, Liane Galanti, Avi Yaeli, and Segev Shlomov. AgentFixer: From failure detection to fix recommendations in agentic systems. In *Proceedings of the 2026 International Workshop on Agentic Engineering*, pp. 96–103. Association for Computing Machinery, 2026. doi: 10.1145/3786167.3788427. URL https://doi.org/10.1145/3786167.3788427.

OpenAI. Introducing SWE-bench Verified, 2024. URL https://openai.com/index/introducing-swe-bench-verified/.

Tianyue Ou, Wanyao Guo, Apurva Gandhi, Graham Neubig, and Xiang Yue. AgentDiagnose: An open toolkit for diagnosing LLM agent trajectories. In Ivan Habernal, Peter Schulam, and Jörg Tiedemann (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 207–215, Suzhou, China, November 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-demos.15. URL https://aclanthology.org/2025.emnlp-demos.15/.

Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses, 2026. URL https://arxiv.org/abs/2603.25723.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023. URL https://arxiv.org/abs/2305.15334.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs, 2023. URL https://arxiv.org/abs/2307.16789.

Alfin Wijaya Rahardja, Junwei Liu, Weitong Chen, Zhenpeng Chen, and Yiling Lou. Can agents fix agent issues?, 2025. URL https://arxiv.org/abs/2505.20749.

Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William E. Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Kenji Toyama, Robert James Berry, Divya Tyamagundlu, Timothy P. Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=il5yUQsrjC.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://proceedings.neurips.cc/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.850. URL https://aclanthology.org/2024.acl-long.850/.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. URL https://arxiv.org/abs/2305.16291.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023. URL https://arxiv.org/abs/2308.08155.

Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4All: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, pp. 1–13. ACM, 2024. doi: 10.1145/3597503.3639121. URL https://doi.org/10.1145/3597503.3639121.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html.

Xiaofei Xie, Lei Ma, Felix Juefei-Xu, Hongxu Chen, Minhui Xue, Bo Li, Yang Liu, Jianjun Zhao, Jianxiong Yin, and Simon See. DeepHunter: Hunting deep neural network defects via coverage-guided fuzzing, 2018. URL https://arxiv.org/abs/1809.01266.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '11, pp. 283–294. ACM, 2011. doi: 10.1145/1993498.1993532. URL https://doi.org/10.1145/1993498.1993532.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the LLM agentic systems?, 2025a. URL https://arxiv.org/abs/2509.03312.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026a. URL https://openreview.net/forum?id=pUpzQZTvGY.

Lechen Zhang, Tolga Ergen, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. SPRIG: Improving large language model performance by system prompt optimization. In *The Fourteenth International Conference on Learning Representations*, 2026b. URL https://openreview.net/forum?id=VdVV24KSWK.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 76583–76599. PMLR, 2025b. URL https://proceedings.mlr.press/v267/zhang25cq.html.

Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement, 2024. URL https://arxiv.org/abs/2404.05427.

A Terminology, Acronyms, and Naming Rationale

This appendix defines the short names used in the paper and explains why the method-specific names match their role in the framework. The explanations below describe only the terminology used in this paper.

Table 4: Acronyms and short names used in the paper.

Short name	Full name	Meaning in this paper
LLM	**Large Language Model**	The base language model used by the agent; model parameters are outside the harness repair target.
API	**Application Programming Interface**	A callable software interface exposed to the agent or used internally by the harness.
IHT	**Invariant Harness Tomography**	The diagnosis core that reconstructs latent harness mechanisms from complementary behavioral, invariant, source, and intervention evidence.
Proactive IHT	**Proactive Invariant Harness Tomography**	IHT together with targeted evidence acquisition: when the observed execution is non-separating, the method actively creates additional execution views before promoting a mechanism to repair.
HTIR	**Harness-aware Trace Intermediate Representation**	A structured trace representation that normalizes runtime steps and aligns execution evidence with harness artifacts.
HFS	**Harness Fault-Signal Signature**	A compact summary of fault-relevant runtime signals, including task outcome, surfaced errors, retry/termination behavior, truncation, reported status, and state/effect evidence.
AS	**Adaptation Signature**	A compact summary of how the agent reacts to harness-side difficulty, such as retries, tool switching, pagination, fallback behavior, or workarounds.
OSHF	**Outcome-Silent Harness Flaw**	A coarse corpus-level flag for flaws whose final task outcome does not itself expose the problem. It is auxiliary evidence rather than an observability-level label.
L1/L2/L3	**Observability Levels 1–3**	L1: a harness bug causes a task-failure trajectory; L2: the task succeeds but the bug leaves abnormal trajectory evidence; L3: the ordinary trajectory is healthy because the bug is not triggered, and a different reachable condition is needed to expose it.
CR@K	**Candidate-anchor Recall at K**	File-level localization metric: whether at least one verified fix file appears among the top-K ranked source anchors.
H_0	**Initial harness**	The unmodified benchmark harness from which closed-loop repair begins.
HF	**HarnessFix**	Short table label for the HarnessFix baseline.
TCR	**Task Completion Rate**	Fraction of benchmark tasks completed according to the official evaluator.
RQ	**Research Question**	Prefix used for the four evaluation questions, RQ1–RQ4.
PR	**Pull Request**	A repository change proposal used as one source of issue/fix evidence.

Short name	Full name	Meaning in this paper
CI	**Confidence Interval**	Statistical uncertainty interval; proportion plots use 95% Wilson intervals unless stated otherwise.
JSON	**JavaScript Object Notation**	Structured serialization format used for diagnosis-agent outputs and saved artifacts.
CSV	**Comma-Separated Values**	Tabular serialization format used for supplementary per-case results.

A.1 Why "Proactive Invariant Harness Tomography"?

The name describes the method at four levels. *Invariant* refers to runtime obligations that should hold independently of the final task score, such as preserving tool errors, respecting approval boundaries, and maintaining state and verification contracts. These invariants provide structured evidence, but they are not treated as complete specifications of correctness. *Harness* identifies the object being diagnosed and repaired: the runtime infrastructure around the base model, including context construction, tool interfaces, orchestration, state, observability, verification, and governance. *Tomography* captures the central diagnostic idea: the responsible mechanism is latent, while each available trajectory, invariant finding, source anchor, and intervention response provides only a partial projection of it. IHT is the multi-view inference principle that combines the evidence views available for a case to narrow the mechanism version space. Passive IHT uses only naturally available views; Proactive IHT acquires additional behavioral views through activation and verification interventions when the ordinary execution is insufficient. Finally, *Proactive* denotes the evidence-acquisition step: when the ordinary execution does not separate a healthy harness from a defective one, the method deliberately runs a bounded probe to obtain an additional behavioral projection before repair.

A.2 Why HTIR, HFS, and AS?

Harness-aware Trace Intermediate Representation (HTIR) emphasizes that the representation is more than a flat log. "Trace" denotes the runtime sequence, "Intermediate Representation" denotes a normalized form used by later diagnosis stages, and "Harness-aware" denotes explicit links between runtime evidence and the harness artifacts that shape it. In this paper HTIR is the structural view of execution.

Harness Fault-Signal Signature (HFS) summarizes *what the harness exposed*. "Fault-signal" is intentionally broader than task failure: a signal can be a surfaced error, an inconsistent status, a retry pattern, a truncation marker, a missing state effect, or the absence of expected evidence on an otherwise successful run. "Signature" denotes a compact evidence profile, not a unique hash or a causal verdict. HFS therefore remains meaningful across L1, L2, and healthy/L3-candidate executions.

Adaptation Signature (AS) summarizes *how the agent responded*. Examples include retrying, switching tools, paginating, falling back to another path, or working around a harness-side limitation. AS complements HFS: two runs can expose similar harness signals but differ in whether and how the agent compensates. The pair helps diagnosis distinguish a harness symptom from the agent behavior that follows it.

A.3 Why Outcome-Silent Harness Flaw and the three observability levels?

Outcome-Silent Harness Flaw (OSHF) is an auxiliary corpus concept: it asks only whether the final task outcome exposes the flaw. This is deliberately coarser than the three-level observability taxonomy because both L2 and L3 can preserve the final task outcome while differing in trajectory evidence. We therefore use OSHF only as a supporting corpus feature, not as the level oracle.

The three observability levels are ordered by what an ordinary execution reveals, not by defect severity. **L1** is a bug-induced task failure. **L2** preserves the task outcome but exposes the bug through abnormal trajectory evidence; the invariant library is designed to detect such runtime violations. **L3** is different: the ordinary task does not activate the defect, so the trajectory remains healthy and a different reachable execution condition is required to expose it. A clean successful run is therefore only a healthy/L3 candidate until activation supplies evidence that a latent defect is present.

A.4 WHY CR@K?

Candidate-anchor Recall at K (CR@K) measures source localization under ranked uncertainty. The diagnoser is allowed to retain several plausible mechanisms, so the metric asks whether any verified fix file appears among the first K candidate anchors rather than forcing a single guess.

B INVARIANT REGISTRY AND ALL-TRAJECTORY AUDIT

The code snapshot supplied with this paper contains 40 registered harness invariants: 20 universal rules, 16 code-derived rules, and 4 repository-derived rules. Thirty rules are enabled and active for routing, six universal rules are enabled but informational (they can support diagnosis without independently changing the L1/L2 routing decision), and four repository-derived rules are registered but disabled in the default routing configuration to avoid leaking benchmark-specific labels. By harness layer, the 40 rules comprise governance (8), tooling (8), context (8), lifecycle (6), observability (5), verification (3), and execution (2).

Routing contract used in RQ2

L1: the product/task oracle fails, regardless of additional invariant evidence.
L2: the task succeeds and at least one active invariant violation remains after the frozen false-positive filter.
Healthy/L3 candidate: the task succeeds and no active violation remains. This is a routing state, not proof of a latent defect.
Unknown/error: missing product outcomes or infrastructure/checker failures are not admitted as healthy evidence.

B.1 COMPLETE 40-RULE REGISTRY

Table 5 lists every rule in the provided implementation. "Active" means the rule is enabled and may participate in routing. "Informational" means the rule is enabled but its current checker is intentionally not allowed to create an L2 label by itself. "Disabled repo-derived" denotes rules retained as repository-specific evidence templates but disabled in the default benchmark configuration.

Table 5: Complete invariant registry used by the current implementation.

Invariant ID	Layer	Mode	Runtime obligation	Violation signal
`no_internal_reasoning_leak`	Governance	Active	Internal reasoning or thinking must remain separated from user-visible assistant output.	Reasoning or thinking markers appear in user-visible assistant content.
`no_swallowed_tool_error`	Tooling	Informational	Tool failures must remain observable or be explicitly handled; they must not be silently treated as success.	A tool error occurs but leaves no error trace or corrective action.
`tool_call_arguments_valid`	Tooling	Informational	Tool calls must satisfy the declared argument contract and parsing failures must be surfaced.	Required arguments are missing, undeclared arguments appear, or formatting is invalid without feedback.
`tool_success_status_consistent`	Tooling	Informational	Reported tool success/status must agree with the real execution effect.	Reported status conflicts with the observed execution effect.

Invariant ID	Layer	Mode	Runtime obligation	Violation signal
`no_silent_context_truncation`	Context	Active	Context or observations may be truncated only with an explicit truncation marker; critical evidence must not disappear silently.	Long content is clipped without a visible marker.
`no_cross_task_state_leak`	Context	Informational	Task/session state and memory must remain isolated across tasks or conversations.	State from a previous task appears in a later task.
`no_stale_context_as_current`	Context	Informational	Stale context must not be presented or consumed as current state.	Outdated information is used as the current state.
`no_premature_hang_completion`	Lifecycle	Active	Completion/termination requires the task-specific completion preconditions; the harness must not finish or hang prematurely.	The run terminates or hangs before required outputs/effects are complete.
`retry_policy_respected`	Lifecycle	Active	Retry behavior must respect retry budgets, cost limits, and stopping conditions.	Repeated actions exceed the retry budget or form an unbounded loop.
`required_checkpoint_not_skipped`	Lifecycle	Active	Required orchestration checkpoints, confirmations, approvals, or prerequisite stages must not be skipped.	A required checkpoint is absent from the executed workflow.
`trace_evidence_sufficient`	Observability	Active	Execution should retain enough structured trace and harness metadata to make important runtime events diagnosable.	Critical messages or harness metadata needed for diagnosis are missing.
`reported_status_consistent`	Observability	Informational	Harness-reported run status must agree with the real runtime state.	A runtime error is formatted or reported as success.
`usage_cost_recorded`	Observability	Active	Token/cost usage should be recorded so budget and audit information remain traceable.	The run has no recoverable usage information.
`verification_not_bypassed`	Verification	Active	Required validators/tests must execute before a critical result is accepted or submitted.	Completion is declared without the required verification step.
`verification_result_not_ignored`	Verification	Active	A failed verification result must affect subsequent control flow.	The harness proceeds or submits after verification has failed.
`no_secret_metadata_exposure`	Governance	Active	Secrets and privileged metadata must not appear in user-visible output, logs, or error text.	Output contains credential/token/secret patterns.
`high_impact_action_approved`	Governance	Active	High-impact actions require explicit approval or an applicable policy grant.	Deletion, reset, privilege escalation, or similar high-impact actions occur without approval evidence.

Invariant ID	Layer	Mode	Runtime obligation	Violation signal
restricted_action_policy	Governance	Active	Restricted actions must obey allow/prompt/forbidden policy semantics.	A policy-forbidden action executes or a prompt-required action executes without approval.
sandbox_boundary_kept	Execution	Active	Execution must stay within the intended sandbox/file/network/privilege boundary.	The run exhibits an unblocked sandbox escape or out-of-scope privileged action.
action_effect_matches_state	Execution	Active	Reported mutations and action effects must agree with the environment state after execution.	The expected state change is absent or the reported change differs from real state.
cd_swe_clipped_observation_marker	Context	Active	SWE-agent observations that are clipped must carry the explicit response-clipped marker.	A long observation is clipped without the marker.
cd_swe_single_tool_call	Tooling	Active	A single SWE-agent response should contain exactly one tool call under its parser contract.	One assistant response contains multiple tool calls.
cd_swe_required_args_present	Tooling	Active	SWE-agent tool calls must include required arguments.	A required argument is missing and the call is silently executed.
cd_swe_retry_budget	Lifecycle	Active	SWE-agent retry behavior must respect the configured cost limit.	Retries continue beyond the configured budget.
cd_codex_secret_redaction	Governance	Active	Codex output must pass the configured secret-redaction patterns.	OpenAI, AWS, Bearer, or assignment-style secrets remain unredacted.
cd_codex_execpolicy_decision	Governance	Active	Codex command execution must respect Allow/Prompt/Forbidden exec-policy decisions.	Forbidden commands run or Prompt commands run without approval.
cd_codex_apply_patch_valid	Verification	Active	Codex patch application must pass apply-patch format/context validation.	Patch application fails but is treated as success.
cd_opencode_patch_markers	Tooling	Active	OpenCode patches must contain paired Begin Patch and End Patch markers.	A patch is missing either required marker.
cd_opencode_default_deny	Governance	Active	OpenCode agents without an explicit permission rule should default to deny for privileged actions.	A high-impact action executes despite absent permission configuration.
cd_opencode_compaction_no_loss	Context	Active	OpenCode context compaction/generation changes must not silently remove critical evidence.	Critical evidence disappears across compaction.
cd_cline_high_impact_approval	Governance	Active	Cline high-impact tools (writes, commands, deletion) require user approval by default.	A high-impact tool executes without approval evidence.

Invariant ID	Layer	Mode	Runtime obligation	Violation signal
cd_cline_max_consecutive_mistakes	Lifecycle	Active	Cline must stop when the configured consecutive-mistake threshold is reached.	Execution continues after the mistake limit.
cd_cline_session_exportable	Observability	Active	Cline session history should remain exportable and auditable.	Session history cannot be recovered/exported.
cd_claude_memory_isolation	Context	Active	Claude-style memory retrieval should apply relevance/aging and maintain cross-session isolation.	Irrelevant, stale, or cross-session memory is injected.
cd_claude_token_budget	Context	Active	Claude-style harnesses should check token budget before issuing the next model call.	The run proceeds after exceeding the configured context/token budget.
cd_claude_cost_tracked	Observability	Active	Per-call cost should be tracked.	A model call has no associated cost record.
rd_tool_interface_contract	Tooling	Disabled repo-derived	Repository tool usage/configuration must agree with documented tool-interface contracts.	Tool use or configuration violates the repository-specific interface contract.
rd_llm_output_handling	Tooling	Disabled repo-derived	Model output and context-length handling must follow the repository-specific output contract.	Malformed output or context-length failure is silently accepted/mishandled.
rd_memory_content_integrity	Context	Disabled repo-derived	Repository memory initialization and content must preserve integrity.	Memory is missing, corrupted, incorrectly initialized, or written incorrectly.
rd_workflow_orchestration	Lifecycle	Disabled repo-derived	Repository workflow ordering, branching, parallelism, and joins must follow the intended orchestration contract.	Executed workflow structure violates the repository-specific orchestration design.

B.2 Calibration and frozen false-positive filters

Two invariants can over-fire on short observability trajectories. The affected rules are `no_premature_hang_completion` and `required_checkpoint_not_skipped`. RQ2 therefore applies a fixed false-positive filter to these families before assigning a trajectory level, and the same filter is used for every reported RQ2 case. Section 7 reports the resulting downside: the conservative filter removes real L2 evidence in several cases, causing L2-to-L3 confusions. We keep this fixed rule rather than applying case-specific tuning.

B.3 Why invariants are not bug detectors

An invariant violation is evidence that a runtime obligation was broken; absence of a violation does not prove the source is correct. Likewise, source/coverage risk does not itself establish a defect. The method therefore uses invariants for routing and candidate evidence, while causal verification is deferred to active probes. The four disabled repository-derived rules are especially important in this

regard: they are useful as descriptive contracts during analysis but are not allowed to act as hidden ground truth during the benchmark runs.

C Multi-Evidence Diagnosis and Mechanism Version Space

C.1 Evidence views

The diagnosis stage compiles complementary runtime evidence and lets an agent iteratively inspect the method-visible harness source. Candidate mechanisms are maintained as a dynamic version space rather than forced into a fixed graph structure.

Table 6: Evidence views consumed by the source-grounded diagnoser.

View	Information supplied to diagnosis
HTIR	Harness-aware execution steps, dependencies, actors, inputs/outputs, exceptions, and source/layer anchors.
HFS	Product outcome, reported tool/status behavior, exposed/swallowed errors, retries, truncation markers, and state/effect signals.
AS	Agent adaptation such as tool/API switches, retry-after-damage, pagination, bypass/workaround, or no adaptation.
Invariant evidence	Violated contract IDs and supporting steps, especially for successful but abnormal L2 trajectories.
Activation evidence	Probe condition, manifested symptom, state/effect delta, and execution record for an activated healthy candidate.
Raw trajectory	Bounded original messages/tool calls retained as a fallback when the structured views omit useful context.

C.2 Agentic source-grounded diagnosis

The diagnoser uses a shell-capable agent over the method-visible buggy source snapshot. It may repeatedly search symbols/keywords, read candidate files, connect evidence to runtime code, and revise its explanation. The output is a ranked set rather than a single forced cause:

```
mechanism, defect_class, harness_layer,
source_anchor / target_files,
trigger_condition, expected_manifestation,
confidence, supporting_evidence
```

We denote this dynamic set by $\mathcal{M}_v = \{m_1, \ldots, m_k\}$. The intended semantics are "mechanisms still compatible with current evidence," not "multiple patches to try." A candidate can be ruled out, supported, or causally verified by later probes.

Why HFS/AS are complementary rather than causal proof

HTIR answers primarily *where and how execution evolved*. HFS summarizes *what symptom the harness exposed*; AS summarizes *how the agent reacted*. Two mechanisms can yield similar terminal trajectories but differ in tool status, state effects, or recovery behavior. These signatures therefore sharpen the candidate set, but the final causal claim still requires an intervention.

C.3 Causal mechanism verification

Activation and mechanism verification are distinct probe families. An *activation probe* is used only when ordinary execution is healthy and asks whether missing evidence can be created. After an anomaly exists, a *mechanism-verification probe* asks which $m_i \in \mathcal{M}_v$ explains it. A singleton candidate still receives a confirmation probe; multiple candidates receive a disambiguation probe. If the probe budget ends without identification, the system abstains and does not invoke repair.

D Operational Details of Proactive IHT

D.1 Method-visible versus evaluator-only artifacts

The benchmark physically distinguishes method inputs from historical ground truth.

Method-visible	**Evaluator-only gold**
Current buggy source snapshot; current trajectory/HTIR; HFS/AS; active invariant evidence; benign execution coverage; general trigger patterns; generic probe executor.	Target issue/PR text; fixed revision and developer diff; gold location; gold trigger; historical reproducer; historical symptom matcher; verifier-only metrics not present in the trajectory.

The fixed revision is touched only after the method has selected an activation/verification probe or proposed a repair. This prevents developer knowledge from becoming a shortcut for diagnosis.

D.2 Controlled activation boundary

For model-output-driven triggers, a local OpenAI-compatible scripted endpoint can deterministically produce missing/wrong-type arguments, malformed calls, empty responses, hallucinated tools, streaming responses, and related conditions. Separate adapters control context length, runtime/configuration, permissions, retry boundaries, and cross-session state. Only the boundary condition is controlled; the harness parser, dispatcher, lifecycle logic, state handling, and verifier remain the real historical implementation.

D.3 Repair acceptance

A repair is generated only from a causally verified mechanism. The final acceptance gate replays the mechanism-verification probe; for activated L3 it also replays the original activation probe. All benign tasks must preserve task outcomes, active invariants must be satisfied, and non-target regression checks must pass. A semantically correct patch need not textually match the developer fix.

D.4 Limitations and scope

The 9,058-record empirical corpus is derived from public issue/PR evidence and remains subject to repository and reporting bias. Observability is relative to the task bank and available oracles; finite probes cannot prove defect absence. ObservabilityBench intentionally favors reproducible Python harness defects and is not a population estimate. RQ2 also exposes an invariant-calibration limitation on L2: filtering rules that suppress false positives on short traces can remove true anomaly evidence. Controlled model adapters improve causal reproducibility but do not reproduce the full behavior distribution of real LLMs. We therefore treat unresolved cases and non-manifestation conservatively and separate historical-mechanism evaluation from real-model/public-benchmark robustness.

E Empirical Corpus and Annotation Protocol

E.1 Mining funnel

We mine public GitHub issues and merged pull requests from 16 open-source agent projects. The raw collection contains 15,294 rows representing 15,293 unique URLs. We deduplicate by URL and remove obvious feature requests, questions, documentation-only changes, announcements, refactors, and other non-bug records using deterministic title-based filters, leaving 12,952 candidates. The complete funnel is therefore 15,293 unique records $\rightarrow$ 12,952 filtered candidates $\rightarrow$ 9,058 harness-related records $\rightarrow$ 9,045 records with a defined observability level.

E.2 Stage 1: harness-related filtering

The first LLM stage determines whether a candidate concerns the agent harness rather than a dependency, installation problem, or unrelated user-interface issue. RQ1 was run before DeepSeek-V4.1-Flash was released; DeepSeek-V4-Flash is therefore the primary classifier and annotator for this

corpus. For each candidate it receives the repository/harness name, record type, title, and issue/PR body and returns: (i) `is_harness_related`; (ii) a primary harness layer; (iii) an Outcome-Silent Harness Flaw (OSHF) candidate flag; (iv) a mechanism-family hint; (v) a free-text trigger condition; and (vi) confidence. The harness layers cover tooling, context/memory, lifecycle/orchestration, observability, verification, governance/security, and execution/sandbox. We retain the 9,058 records classified as harness-related for the second stage.

The OSHF flag is deliberately coarse and is not used as the L1/L2/L3 oracle. It only asks whether the reported flaw could plausibly remain hidden from the final task outcome. The final observability level is derived in Stage 2 from task impact and trajectory evidence.

E.3 Stage 2: observability annotation

The second LLM stage again uses DeepSeek-V4-Flash as the primary model. Its input contains the harness name, record type, title, the bounded issue/PR description used by the pipeline, and an available fix diff summary when the GitHub record is a pull request. The annotator predicts five substantive axes plus reproduction evidence:

- **Trigger**: unconditional, edge input, stateful, scale, timing, or adversarial.
- **Manifestation**: explicit error/crash, subtle wrong output, silent internal effect, or suspended/hanging behavior.
- **Task impact**: `fail` or `preserved`.
- **Trace evidence**: `visible` or `silent`.
- **Impact type**: crash/error, wrong output, data loss, state corruption, security leak, hang/stall, silent no-op, resource waste, or unknown.
- **Discovery/reproduction evidence**: for L3 records, explicit steps, linked test/PR, reported usage, partially specified condition, or unknown.

The prompt explicitly allows `unknown` instead of forcing an unsupported judgment. In the final annotation file, 9,032 harness-related records are annotated by DeepSeek-V4-Flash, 20 use the DeepSeek-V4-Pro fallback after the Flash response cannot be parsed, 13 remain undefined after both attempts, and one record has incomplete model metadata. These 13 undefined records are excluded from the level-distribution denominator.

E.4 Level decision rule

The issue-level label is derived from the two axes that directly encode observability:

- **L1**: the reported harness bug causes task failure (`task_impact=fail`).
- **L2**: the task still succeeds, but the bug leaves abnormal evidence in the trajectory (`task_impact=preserved` and `trace_evidence=visible`). The invariant library used by Proactive IHT operationalizes this kind of trajectory-level anomaly at runtime.
- **L3**: the ordinary task succeeds and its trajectory is healthy because the bug is not triggered by that execution (`task_impact=preserved` and `trace_evidence=silent`). A different reachable condition is required to activate and expose the defect.

For the later executable suite, L3 is checked more strictly at runtime: the buggy revision must pass the frozen benign execution with a healthy trace, while a separate reachable activation condition produces the target manifestation and the fixed revision does not.

E.5 Independent two-author manual audit

We sample 100 records from each LLM-assigned level. Two authors independently inspect the underlying issue/PR evidence against the same observability definitions used by the annotation prompt; they do not judge correctness from the model rationale alone. We use a conservative confirmation rule: an LLM label is counted as confirmed only when both authors accept it. Both authors confirm 100/100 L1, 95/100 L2, and 96/100 L3 labels (291/300 overall). Both reject 0/100

L1, 3/100 L2, and 1/100 L3 labels; the remaining 0, 2, and 3 records, respectively, are auditor disagreements.

Because the unconditional–L3 cell is easy to misread as contradictory, we conduct a second targeted audit of 100 records sampled from that cell. Both authors confirm 91/100 labels, both reject 4/100, and they disagree on 5/100. Here *unconditional* means that once the affected harness behavior is exercised, no additional edge input, state, timing, or adversarial condition is required; an ordinary task can still remain L3 by never reaching that behavior. The audits support corpus-level analysis while remaining distinct from the executable replay criteria used in RQ2/RQ3.

E.6 Reproduction evidence within L3

Appendix Figure 3 summarizes how much explicit replay evidence is available within L3-labeled reports. Most records do not provide a deterministic reproducer: only 22.2% contain either explicit steps or a linked test/PR, while the remainder are reported from real usage or leave the discovery path underspecified.

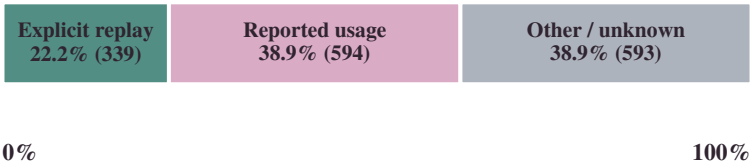
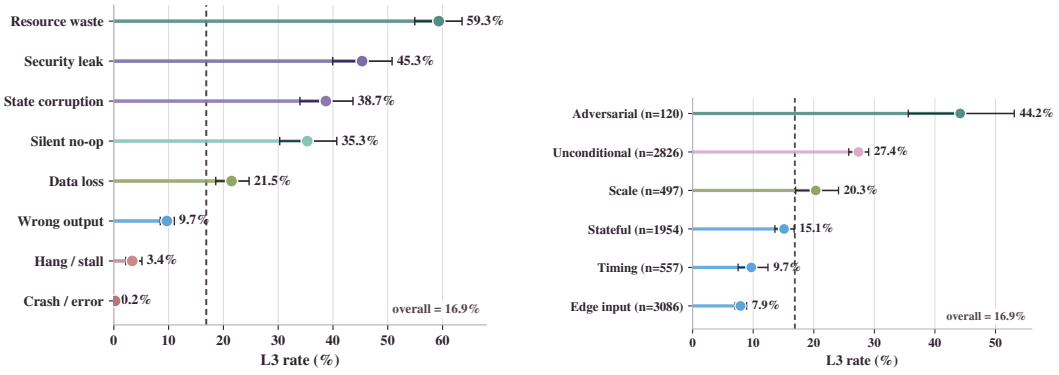


Figure 3: Reproduction evidence available within L3-labeled issue/PR records.

E.7 Additional L3 stratified distributions

For main-text economy, Figure 2 displays the overall distribution and the L3 layer view. The two remaining L3 stratifications are reported here with the same category denominators, Wilson 95% intervals, ordering, palette, and dashed overall-rate reference.



(a) L3 rate by impact category

(b) L3 rate by trigger type

Figure 4: Additional L3 stratifications in the LLM-assisted, manually audited corpus. Whiskers are Wilson 95% confidence intervals; dashed lines mark the 16.9% corpus-wide L3 rate.

E.8 Per-harness prevalence

Figure 5 reports L3 rates across the 16 projects. The dashed line marks the 16.9% corpus-wide rate. We use these project-level numbers to establish breadth, not to rank harness quality: issue selection, project maturity, and reporting practices differ substantially.

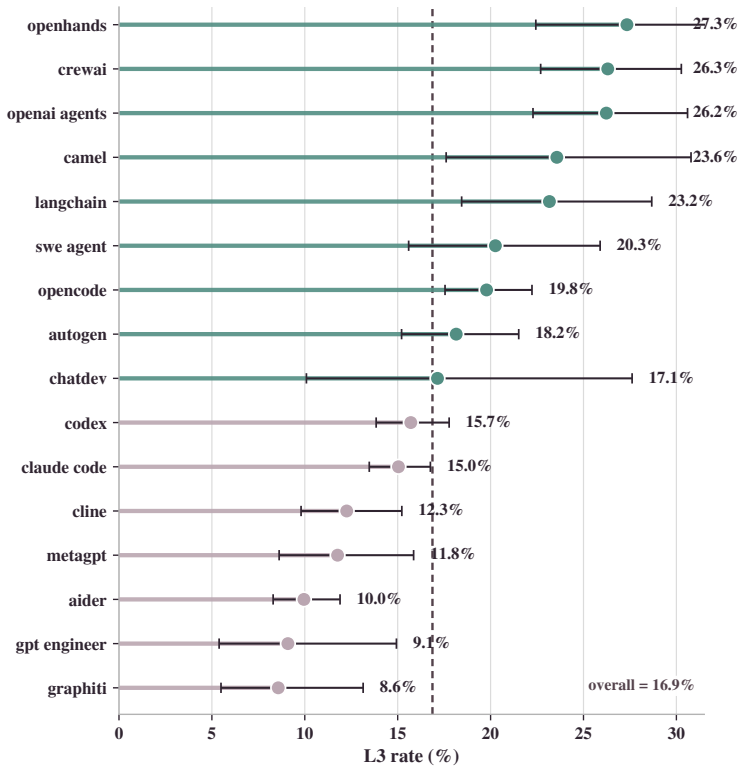
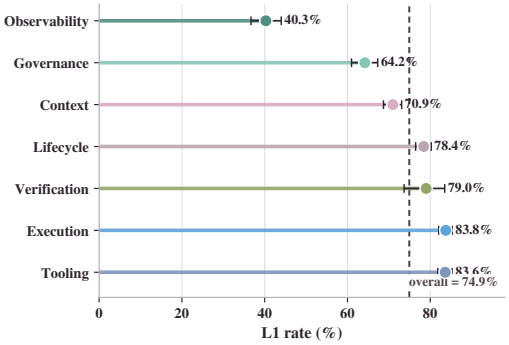


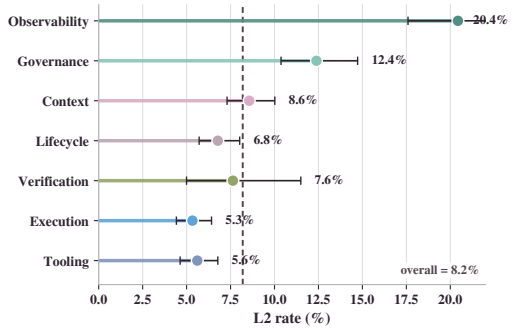
Figure 5: L3 rate by harness in the LLM-assisted, manually audited corpus. All 16 projects contain non-zero trajectory-silent mass; rates range from 8.5% to 27.3%.

E.9 L1 AND L2 STRATIFIED DISTRIBUTIONS

The main text focuses on L3 because it motivates proactive evidence acquisition. For completeness, Figures 6–9 report the corresponding predictive L1 and L2 rates using the same category denominators, Wilson 95% intervals, ordering, palette, and dashed overall-rate reference as the L3 plots. These views are useful for checking that the L3 patterns are not artifacts of hiding the other two levels.



(a) L1 rate by harness layer

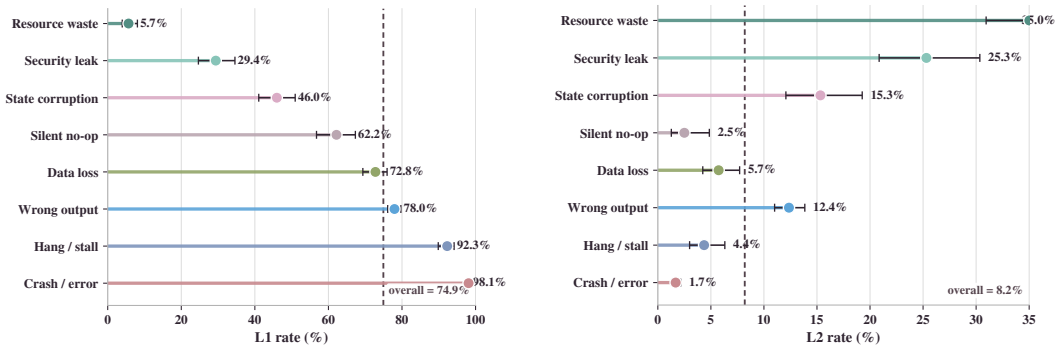


(b) L2 rate by harness layer

Figure 6: LLM-assisted L1/L2 rates by harness layer. Whiskers are Wilson 95% confidence intervals; dashed lines mark the corresponding corpus-wide level rate.

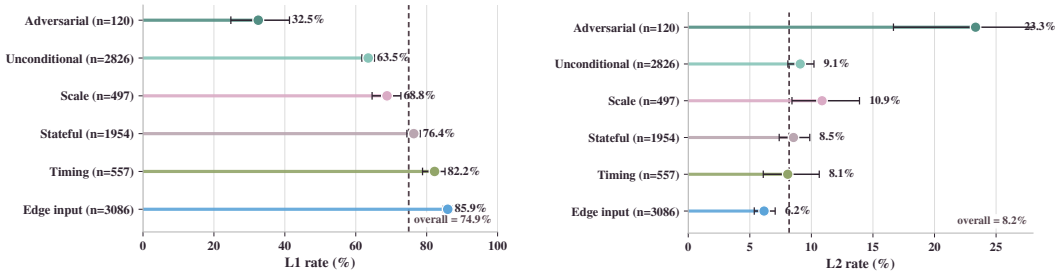
E.10 MECHANISM-FAMILY DIAGNOSTIC

Mechanism-family labels are used as a secondary corpus diagnostic rather than as a main result. Among sufficiently populated categories, L3 rates are high for approval-bypass (27.1%) and verification-bypass (26.1%), followed by context-drop (18.4%) and stale-state (17.2%); tool-schema



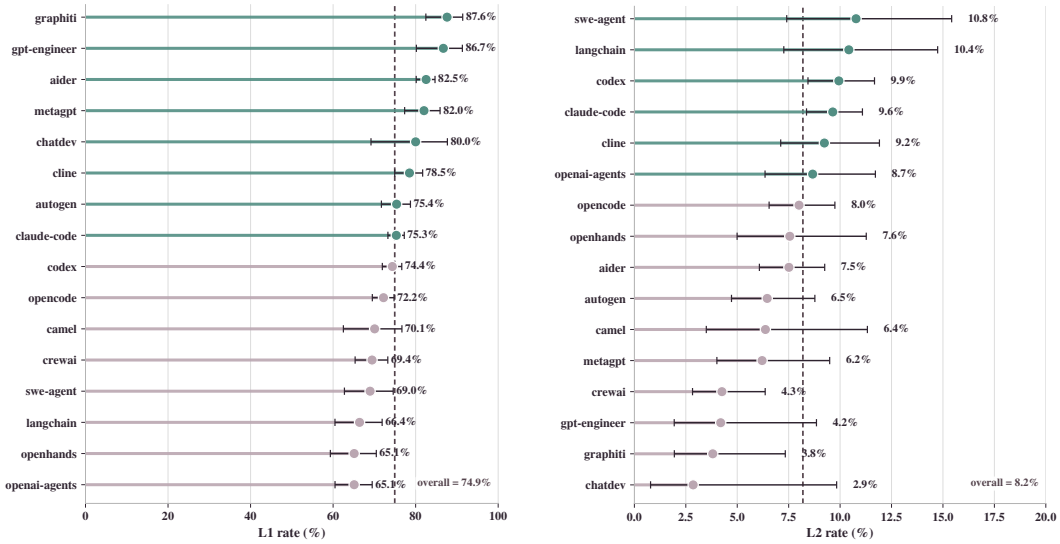
(a) L1 rate by impact category (b) L2 rate by impact category

Figure 7: LLM-assisted L1/L2 rates by impact category, using the same known-impact categories as the main-text L3 panel.



(a) L1 rate by trigger type (b) L2 rate by trigger type

Figure 8: LLM-assisted L1/L2 rates by trigger type. Labels include the per-trigger sample size.



(a) L1 rate by project (b) L2 rate by project

Figure 9: LLM-assisted L1/L2 rates across the 16 projects. As with the L3 project plot, these numbers establish breadth and should not be interpreted as project-quality rankings.

(6.6%) and retry (6.3%) are much more often visible. This pattern is consistent with the main layer analysis: failures in policy, verification, context, and persistent state can preserve the immediate task path, whereas schema and retry errors frequently surface directly.

E.11 OSHF-CANDIDATE CONSISTENCY

The Outcome-Silent Harness Flaw (OSHF)-candidate flag is intentionally coarse: it asks whether the final task outcome alone exposes the reported flaw, whereas the L1/L2/L3 label also uses trajectory evidence. Of 4,451 OSHF candidates, 860 are annotated L3, while 666 of the 1,526 annotated L3 records are outside the OSHF screen. This partial overlap is expected because OSHF and the three-level taxonomy answer different questions. We therefore use OSHF only as an auxiliary corpus feature; observability levels are derived from the task-impact and trace-evidence axes and executable cases are validated independently.

F OBSERVABILITYBENCH CONSTRUCTION AND FULL RQ2 PROTOCOL

F.1 FROM A HISTORICAL REPORT TO AN EXECUTABLE CASE

The large RQ1 corpus is useful for prevalence but insufficient for method evaluation because issue text does not guarantee that a defect can be replayed. We therefore construct a smaller executable historical suite from real reports. The construction pipeline is intentionally conservative:

1. **Historical mapping.** Identify the public issue/PR, buggy revision H^-, fixed revision H^+, and the harness-core root cause. Reverse-patching is allowed only to reconstruct the historical buggy state when the repository history makes the parent revision inconvenient; the resulting behavior must still match the historical symptom.
2. **Environment reconstruction.** Create isolated buggy/fixed worktrees under a shared dependency/runtime specification and verify that both can execute the case driver.
3. **Driver construction.** Implement a normal `benign` mode for every case and one level-specific trigger mode: `fail` (L1), `anomaly` (L2), or `probe` (L3). The driver exercises the real harness code rather than mutating the implementation.
4. **Dual verifier.** A verifier separately returns a *product contract* (did the user-visible task complete?) and a *norm contract* (did the harness satisfy the relevant runtime obligation?). Verifier-only metrics are kept outside the method-visible trajectory.
5. **Paired validation.** Re-run the same mode on H^- and H^+. The fixed revision must eliminate the historical manifestation without breaking the benign execution.

Executable level contract

L1: benign H^-/H^+ pass; trigger H^- fails the product contract and exposes the historical failure; trigger H^+ passes.
L2: benign H^-/H^+ pass; anomaly H^- preserves product success but violates the norm contract in the trajectory; anomaly H^+ preserves both.
L3: benign H^-/H^+ both pass product and norm contracts with healthy traces; a separate probe makes H^- violate the target norm/effect while the same probe remains safe on H^+. Thus the level is defined by the *ordinary* execution, not by the probe trajectory.

Four candidate cases are outside the reported 36-case set because they do not satisfy the fixed admission criteria for source revision and conversational trajectory evidence. The benchmark spans four projects: OpenAI Agents (14), LangChain (8), CrewAI (7), and CAMEL (7). Table 7 provides the level composition removed from the main paper; full revision mappings are listed in Appendix J.

F.2 PER-CASE ARTIFACT SCHEMA

Each historical case entry contains at least:

```
case_id, harness, issue_url, level,
```

Table 7: **ObservabilityBench composition.** Cases are admitted by historical reproducibility rather than by balancing observability levels.

Harness	L1	L2	L3	Total
CAMEL	2	2	3	7
CrewAI	1	0	6	7
LangChain	5	2	1	8
OpenAI Agents	4	4	6	14
Total	**12**	**8**	**16**	**36**

```
buggy_commit, fixed_commit,
driver, verifier, python_environment,
harness_source_roots, status, notes
```

Execution artifacts include the method-visible trajectory JSON and a sibling metrics record consumed only by the verifier. The registry resolves source roots and worktrees generically; RQ2 contains no per-case decision logic in the baseline dispatch. Representative drivers/verifiers in the supplementary artifact include one case from each level.

G RQ2 Detailed Evaluation

G.1 Tasks and ground truth

For each of 36 cases, RQ2 creates two paired decisions and one single-trajectory regime decision.

- **Trigger pair**: A is the level-specific buggy execution and B is the same mode on the fixed revision; the correct answer is A.
- **Benign pair**: A and B are benign executions of buggy/fixed revisions. Both are runtime-healthy under the executed condition, so neither execution contains manifested defect evidence. The correct answer is therefore *neither*. This is the specificity control.
- **Regime classification**: the single buggy input is `fail_buggy` for L1, `anomaly_buggy` for L2, and `benign_buggy` for L3. Using `probe_buggy` for L3 would leak the activation into a task intended to test passive observability and is therefore disallowed.

G.2 Baseline implementations and budgets

Table 8: RQ2 baseline contracts. "Source" denotes access under the method's native mechanism; no method sees fixed source or issue/fix metadata.

Method	Success traces	Invariants	Source	Decision mechanism
Raw LLM	yes	no	context/path	Direct trajectory/source judgment; paired decisions use 3 independent votes.
HarnessFix	native interface is failure-driven	no	agentic	HTIR followed by its source-inspecting diagnosis agent; 40-step budget.
Auditor	yes	yes	retrieved snippets	Product outcome + frozen invariant checks; healthy traces use HTIR/source judgment of latent risk.
Passive IHT	yes	yes	source-grounded	HTIR + HFS/AS + violations → mechanism candidates; probes disabled.

All model-based calls use DeepSeek-v4.1-Flash with reasoning/thinking disabled. The shared configuration is temperature 0.2, maximum output 4,096 tokens, non-streaming generation, and a 900 s request timeout. These are application programming interface (API) generation settings; method-specific interaction budgets are enforced separately. HarnessFix uses a 40-step source-analysis budget.

The Auditor and Passive IHT use the invariant false-positive filter described in Appendix B, fixed for all reported RQ2 runs.

Information withheld from every RQ2 method

True level; whether A or B is buggy; fixed revision/source; target issue/PR text; developer fix/diff; gold trigger/location; oracle reproducer; verifier-only metrics such as hidden bookkeeping/state counts. These artifacts are used only by the experiment harness to assign ground truth and compute scores.

G.3 Prompt schemas

The exact supplementary scripts contain the full prompts. The key decision schemas are:

Paired discrimination prompt (abridged)

Given two trajectories from the same harness/task under two revisions, decide which execution, if either, contains runtime-supported evidence that a harness defect *manifests under the observed execution condition*. Suspicious source code or a latent defect hypothesis is not by itself manifestation evidence. Return JSON only: {`"chose":"A"|"B"|"neither"|"both", "reason":"..."`}. For a trigger pair, the gold answer is the buggy execution; for a benign pair, the gold answer is *neither*. Auditor/Passive IHT additionally receive their structured evidence and permitted source context.

Regime classification prompt (abridged)

Classify one execution as `L1`, `L2`, `L3`, or `none`: L1 = a harness bug causes task failure; L2 = task success with bug-relevant trajectory abnormality/invariant violation; L3 = the ordinary task succeeds with a healthy trajectory because a latent harness defect is not activated; none = no defect evidence/hypothesis. Return JSON only.

G.4 Aggregate results with Wilson intervals

Table 9: Aggregate RQ2 rates with Wilson 95% intervals. Benign specificity is the fraction of healthy pairs answered *neither*.

Method	Trigger discrimination	Benign specificity	Regime classification
Raw LLM	58% [42,73]	97% [86,100]	58% [42,73]
Auditor	47% [32,63]	97% [86,100]	56% [40,70]
Passive IHT	**67% [50,80]**	14% [6,29]	**78% [62,88]**
HarnessFix	22% [12,38]	3% [0,14]	22% [12,38]

G.5 Level confusion matrices

The regime task reveals why aggregate accuracy alone is insufficient. Passive IHT is maximally sensitive to L3 but maps 7/8 L2 cases to L3; the Auditor maps 5/8 L2 to L3. HarnessFix emits only L1/none by design.

Reporting discipline. Pairwise scoring asks which execution contains manifested runtime evidence, not which source revision is globally defective. For a buggy/fixed trigger pair, the manifested buggy execution should be selected; for a benign buggy/fixed pair, the correct label is *neither*. Benign specificity is computed under this rule. For regime classification, the L3 input is the healthy benign trajectory rather than an activation probe, so RQ2 measures what passive evidence can establish from the ordinary execution.

Table 10: Regime classification confusion counts (rows = gold, columns = prediction).

Method	Gold	Pred L1	Pred L2	Pred L3	Pred none
Raw LLM	L1	11	0	1	0
	L2	0	5	3	0
	L3	1	2	5	8
Auditor	L1	11	0	0	1
	L2	0	1	5	2
	L3	0	0	8	8
Passive IHT	L1	11	0	1	0
	L2	0	1	7	0
	L3	0	0	16	0
HarnessFix	L1	8	0	0	4
	L2	7	0	0	1
	L3	15	0	0	1

H RQ3 AND RQ4 PROTOCOL DETAILS

H.1 RQ3: POST-EXPOSURE SOURCE LOCALIZATION ACROSS THE THREE ORIGIN LEVELS

RQ3 isolates source diagnosis from evidence acquisition. The 36 cases retain their L1/L2/L3 label from the *ordinary* execution, and that gold observability level is supplied to the localization method. Each method is then evaluated on the level-specific execution that exposes the historical defect: `fail` for L1, `anomaly` for L2, and the controlled `probe` witness for L3. Thus an L3 input in RQ3 is not the healthy execution used for regime classification in RQ2. The gold source location, developer fix, and fixed revision remain hidden. This design asks a narrower question: given the level and a behavioral witness of the defect, can the diagnosis stage map the manifestation back to the responsible harness implementation?

Method-visible input. For each case we compile an EvidenceBundle containing the exposed trajectory, HTIR summary, HFS/AS, frozen invariant violations, product outcome, and the set of visible harness-source files. The diagnosis agent receives a sanitized snapshot of the buggy source. The snapshot excludes `.git`, the fixed revision, gold/oracle artifacts, issue text, evaluation outputs, and verifier-only metrics. It may inspect the source with shell commands for at most 20 model calls. DeepSeek-v4.1-Flash is used for every case with reasoning/thinking disabled, temperature 0.2, maximum output 4,096 tokens, non-streaming generation, and a 900 s request timeout. The diagnoser returns a ranked JSON list of two to four candidates with a mechanism description, `failure_nature`, harness layer, confidence, trigger condition, and one or more `source_anchor` paths.

Information withheld in RQ3

Fixed source and revision identity; historical issue/PR text; developer patch/diff; gold source file; verifier-only state; and any evaluator artifact that directly states the root cause. The gold fix is read only after diagnosis to compute file-level localization metrics.

Gold anchors and metric. The gold anchor set is the Python files changed between the verified buggy and fixed revisions inside the case's harness-source roots. Candidate paths are normalized to remove optional function/line annotations, and source-root-compatible prefix differences are normalized before matching. CR@K is one when at least one gold file appears among the first K ranked candidate anchors, and zero otherwise. For a multi-file historical fix, matching any file in the verified gold set counts as a hit. This metric evaluates file-level source localization; it does not assert that every changed file is independently causal.

Matched localization baselines. The Raw-LLM baseline receives the same defect-exposing trajectory, supplied observability level, and method-visible buggy source, but no HTIR/HFS/AS/invariant bundle. The HarnessFix localization baseline receives the exposed trajectory through its HTIR-style representation and the HarnessFix source-inspecting diagnosis stage. Neither baseline sees the gold

file, developer patch, fixed source, or issue/fix metadata. We report aggregate CR@K values for the matched comparison.

Same-suite post-hoc error analysis. The primary diagnosis configuration yields CR@1=11/36, CR@3=17/36, and CR@5=18/36. The same-suite analysis characterizes a recurring attribution error: the mechanism description can identify the relevant behavior while the anchor stops at a generic loop, parser, or adapter rather than the component that defines the behavior. It applies two repository-agnostic constraints to the primary misses. The first continues from a behavior site to the defining component (for example, a model implementation, tool-registration path, event listener, terminator, or state manager). The second requires a concrete implementation file rather than a component directory and identifies the validator or parser relevant to symptom-only behavior. The composed same-suite artifact set has CR@1=14/36, CR@3=23/36, and CR@5=24/36; it is descriptive error analysis rather than an independent benchmark estimate.

The same-suite score is reported only as descriptive error analysis, not as evidence of held-out generalization. It applies attribution constraints to the primary misses and therefore does not estimate behavior on the cases already localized by the primary configuration.

Table 11: RQ3 candidate-anchor recall with Wilson 95% intervals. "Post-hoc" is descriptive same-suite error analysis after the two attribution constraints above.

Setting	CR@1	CR@3	CR@5
Raw LLM + trajectory + source	13.9% [6.1,28.7]	27.8% [15.8,44.0]	33.3% [20.2,49.7]
HarnessFix diagnoser + HTIR + source	25.0% [13.8,41.1]	50.0% [34.5,65.5]	58.3% [42.2,72.9]
Proactive IHT frozen	30.6% [18.0,46.9]	47.2% [32.0,63.0]	50.0% [34.5,65.5]
Proactive IHT post-hoc same-suite	38.9% [24.8,55.1]	63.9% [47.6,77.5]	66.7% [50.3,79.8]

Table 12: Same-suite CR@3 by observability level and repository. Intervals are Wilson 95% intervals; small groups are reported for diagnosis, not ranking.

Level	Hits	CR@3	Repository	Hits	CR@3
L1	10/12	83.3% [55.2,95.3]	LangChain	8/8	100% [67.6,100]
L2	4/8	50.0% [21.5,78.5]	OpenAI Agents	9/14	64.3% [38.8,83.7]
L3	9/16	56.2% [33.2,76.9]	CrewAI	4/7	57.1% [25.0,84.2]
			CAMEL	2/7	28.6% [8.2,64.1]

Execution status and candidate composition. The frozen primary artifact records 35 cases with structured candidates and one parse failure, with 115 structured candidates in total. Its candidate recall values are computed directly from this artifact. The same-suite attribution analysis is reported separately and does not replace the primary result.

What remains difficult. The aggregate comparison shows that HarnessFix is already a strong post-exposure localization baseline: it slightly exceeds frozen IHT at CR@3 and CR@5, while frozen IHT has higher CR@1. The post-hoc IHT refinement improves over both on this same suite, but it should be read as a diagnosis of a reusable attribution failure mode rather than as an independent benchmark win. Thirteen cases remain outside the top three under the post-hoc same-suite configuration. Manual inspection of the saved candidates shows two recurring patterns. First, the diagnoser reaches the correct component family but chooses the wrong provider-specific implementation inside that directory. Second, the candidate explains a symptom such as "plain text was accepted without a tool call" but does not continue to the validator or conversion routine that should have prevented the symptom. These errors motivate the intervention stage of PROACTIVE IHT: a mechanism-targeted rerun can provide evidence that distinguishes nearby source hypotheses, whereas another round of passive source reading may only restate the same symptom. The complete 36-case ranks are reported in Appendix K, and the supplementary comma-separated values (CSV) file preserves full source paths and candidate summaries.

H.2 RQ4: CLOSED-LOOP COMPARISON WITH HARNESSFIX ACROSS PUBLIC BENCHMARKS

We mirror the HarnessFix evaluation protocol on GAIA (Mialon et al., 2024), SWE-Bench Verified (Jimenez et al., 2024; OpenAI, 2024), AppWorld (Trivedi et al., 2024), and Terminal-Bench 2.0 Verified (Merrill et al., 2026). The benchmark/harness pairs and sampled splits are fixed before optimization: open-deep-research with GAIA (60/30/60), mini-swe-agent with SWE-Bench Verified (100/50/100), the official simplified ReAct code agent with AppWorld (90/45/90), and Harbor Terminus-2 with Terminal-Bench 2.0 Verified (34/17/34). These are the same sampled sizes and roles used by HarnessFix (Chen et al., 2026b); they are not reselected based on IHT outcomes.

Table 13: Closed-loop benchmark configuration mirrored from HarnessFix.

Benchmark	Initial harness H_0	Train	Val.	Test
GAIA	open-deep-research	60	30	60
SWE-Bench Verified	mini-swe-agent	100	50	100
AppWorld	official simplified ReAct code agent	90	45	90
Terminal-Bench 2.0 Verified	Harbor Terminus-2	34	17	34

Models and decoding. The matched RQ4 backbones are DeepSeek-v4.1-Flash, MiniMax-M2.5, Qwen3.6-Flash, and HY3. The provider-specific reasoning settings are fixed within each backbone: DeepSeek-v4.1-Flash uses thinking off; MiniMax-M2.5 uses its default reasoning mode; Qwen3.6-Flash uses `enable_thinking=true`; and HY3 uses `reasoning_effort="high"`. All four use temperature 1.0 and a maximum output of 4,096 tokens. Provider-returned model identifiers and complete request parameters are logged with every run. The broader published-baseline comparison additionally uses GPT-5 mini under the HarnessFix setup; only Proactive IHT is evaluated for that table.

Shared repair loop and controlled treatment difference. H_0, sampled tasks, train/validation/test membership, task model, decoding configuration, method-visible source, patch/edit budget, patch audit, validation gate, and official evaluator are held fixed. HarnessFix and PROACTIVE IHT use the same outer train–modify–validate–promote–test loop. The principal treatment difference is evidence acquisition and diagnosis: HarnessFix starts from failed trajectories, whereas PROACTIVE IHT audits all trajectories, compiles HTIR/HFS/AS/invariant evidence, and requires intervention support before a harness-defect mechanism drives repair. In every closed-loop comparison, all L1 and all L2-like training trajectories are analyzed. The L3 branch has a fixed activation budget of two healthy/L3 candidate trajectories per optimization run; evidence-bearing activated trajectories then join L1/L2 in the shared diagnosis stage. This same policy is used in both the four-backbone comparison and the GPT-5-mini comparison. Test trajectories are never used for diagnosis, patch generation, repair selection, or validation.

Promotion and repeated test protocol. A candidate repair must pass syntax/static checks, method-visible scope checks, target-flaw validation, and the shared held-out validation gate before promotion. Output directories are isolated by benchmark, model, method, seed, and harness version. Provider-returned model identifiers are logged and checked to prevent accidental model substitution. The promoted harness and H_0 are then evaluated in three independent test runs; TCR is reported as the arithmetic mean. Offline optimization/repair tokens are reported separately from benchmark task-run cost where the corresponding comparison records them.

Comparison families. RQ4 uses three complementary evaluations and one completed ablation family. (1) A matched HF/IHT grid spans four task backbones and four benchmarks. (2) A GPT-5-mini comparison adds Proactive IHT to the published HarnessFix table containing human-designed and automated evolution/repair baselines; the published non-IHT values are reused rather than rerun. Both closed-loop comparisons use all L1, all L2, and two sampled L3 activations. (3) The AppWorld characterization intentionally removes that L3 budget: it analyzes all L2-like trajectories and activates all healthy/L3 candidates on the 90 training trajectories to study diagnostic capacity and the amount of non-L1 evidence present in a public workload. (4) The Qwen3.6-Flash ablation removes multi-view diagnosis evidence, routing of successful runs, L3 activation, and causal verification across the same

four benchmarks, with three independent held-out test runs per variant. Appendix L contains detailed run-level and provenance tables for all completed RQ4 experiments.

I Proofs and Scope of the Observability Analysis

I.1 Proof of Proposition 1

Let $P_0 = P^P_{\text{clean}}$ and $P_1 = P^P_{\text{def}}$ be the two distributions over behavioral observations. Under equal priors, the Bayes-optimal binary test chooses the hypothesis with larger density (or probability mass) at the observed O. Its success probability is

$$\frac{1}{2}\int \max\{dP_0, dP_1\} = \frac{1}{2}\left(1 + \mathrm{TV}(P_0, P_1)\right). \tag{8}$$

Therefore the minimum error is

$$R^*_P = 1 - \frac{1}{2}\left(1 + \mathrm{TV}(P_0, P_1)\right) = \frac{1 - \Omega_P(H^0, H^1)}{2}. \tag{9}$$

If $\Omega_P = 0$, the induced observation distributions are identical, so $R^*_P = 1/2$. Any deterministic or randomized post-processing of O is also distributed identically under the two hypotheses and cannot improve this error.

What the proposition does and does not cover. The proposition is deliberately about behavioral observations from a fixed execution policy. It does not rule out source-level proofs, developer metadata, static analysis, or a stronger prior over likely bug patterns. Those sources can change a debugger's belief about H_{clean} versus H_{def}. The claim is narrower: when the executed behavior has the same distribution, that behavior carries no additional information for separating the two hypotheses. Our controlled RQ2 study keeps source inspection available precisely to distinguish hypothesis generation from runtime verification.

I.2 Derivation of the passive-exposure relation in Equation 3

Each independent ordinary execution manifests the defect with probability p_d. Hence T follows a geometric distribution,

$$\Pr[T = t] = (1 - p_d)^{t-1} p_d, \tag{10}$$

which gives $\mathbb{E}[T] = 1/p_d$. The probability that none of the first n executions manifests the defect is

$$\Pr[T > n] = (1 - p_d)^n. \tag{11}$$

Requiring at least one manifestation with probability $1 - \delta$ is equivalent to $(1 - p_d)^n \le \delta$. Taking logarithms gives

$$n \ge \frac{\log \delta}{\log(1 - p_d)}, \tag{12}$$

and the minimum integer n is the ceiling of this expression. For small p_d, $\log(1-p_d) = -p_d + O(p_d^2)$, so the leading term is $\log(1/\delta)/p_d$.

Dependence and nonstationarity. The closed form assumes independent executions with a stable manifestation probability. Real agent runs can be dependent because of persistent state, memory, caching, or changing external services. In those settings the geometric model is only an explanatory approximation. The methodological conclusion survives in a weaker form: evidence acquisition should place probability mass on conditions that are more informative than ordinary traffic, and its success must be measured from executed probes rather than assumed from a risk score.

I.3 Connection to probe selection

After a manifestation is available, Proactive IHT maintains candidate mechanisms $M \in \mathcal{M}$. For a proposed probe p with outcome Z_p, an ideal experimental-design objective is

$$\max_{p \in \mathcal{P}} I(M; Z_p \mid E) - \lambda C(p), \tag{13}$$

where E is current evidence and $C(p)$ is execution cost or intervention risk. The implementation does not estimate this mutual information. It uses candidate-specific expected manifestations, source locality, invariant evidence, coverage, and probe scope as practical ranking signals. We include the objective to clarify the role of active verification: a useful probe should make competing mechanisms predict measurably different outcomes.

I.4 SCOPED INTERVENTIONAL SUPPORT

For a candidate harness mechanism m, let a be an activation condition under which the target manifestation is observed, and let I_m be a scoped intervention intended to neutralize or alter only m. Denote target manifestation by $Z(H, a) \in \{0, 1\}$. We call

$$Z(H, a) = 1, \qquad Z(I_m(H), a) = 0 \tag{14}$$

interventional support for m, provided the intervention was actually applied and passes the method's scope audit. This is evidence that m is a causal contributor under the tested condition, not a proof that m is the unique cause. Side effects, incomplete intervention, and interacting mechanisms can all invalidate a stronger causal claim; the implementation therefore also allows CONTRADICTS and UNINFORMATIVE outcomes and keeps unresolved candidates unresolved.

J OBSERVABILITYBENCH CASE INVENTORY

Table 14 lists all 36 reported cases. Commit strings are shortened for readability; the supplementary registry stores the full revision mapping, issue URL, driver, verifier, environment, and source roots.

Table 14: Full ObservabilityBench case inventory.

Case ID	Harness	Level	Buggy rev.	Fixed rev.
camel_4013	camel	L1	00637137	7583e0d6
camel_4181	camel	L1	db2685b7	b25419b3
camel_4037	camel	L2	cc2de7af	5418a10c
camel_4266	camel	L2	ee95406b	a130c891
camel_2605	camel	L3	5a1e0eb2	13051c55
camel_4012	camel	L3	7583e0d6	653a5ff8
camel_4222	camel	L3	ec48f997	2577a704
crewai_3988	crewai	L1	4d8eec96	c59173a7
crewai_4056	crewai	L3	8bd843d1	98ae9b87
crewai_4156	crewai	L3	b9dd166a	1c59ff8b
crewai_4222	crewai	L3	b858d705	703e0f61
crewai_4389	crewai	L3	d4f9f875	67318448
crewai_4788	crewai	L3	cd42bcf0	6d4fcbd7
crewai_5141	crewai	L3	5b6f89fe	270648ba
langchain_34558	langchain	L1	576b6fa0	d6b0c82f
langchain_34910	langchain	L1	72571185	63818b38
langchain_35852	langchain	L1	69a7b9c8	e85c00b9
langchain_35931	langchain	L1	342d8bde	34d122c0
langchain_39749	langchain	L1	2019bf5e	a924e6ca
langchain_37093	langchain	L2	6dcdb63f	d3d5d3ce
langchain_39249	langchain	L2	39b4e0f9	2e4b5ab3
langchain_38840	langchain	L3	576b6fa0	5423f84f
oa_2449	openai-agents	L1	9e812de8	6f31d49f
oa_3961	openai-agents	L1	f663a06a	a335b320
oa_4627	openai-agents	L1	7abe1544	c8b0a928
oa_4655	openai-agents	L1	3603dc92	e773b154
oa_3087	openai-agents	L2	1b7d878b	ae609474
oa_3110	openai-agents	L2	601ecf55	3d1231e0
oa_3863	openai-agents	L2	606fbd0d	efd63ece
oa_4224	openai-agents	L2	c48dd4c1	b6787a3f

Case ID	Harness	Level	Buggy rev.	Fixed rev.
oa_2384	openai-agents	L3	d463a97f	61a2dfd2
oa_2776	openai-agents	L3	8fdb45da	0e88b687
oa_3346	openai-agents	L3	fedc809a	659f3688
oa_4070	openai-agents	L3	4c9e507	76426a37
oa_4322	openai-agents	L3	7bf73afa	ffe7224e
oa_4615	openai-agents	L3	6268f43e	40f0d9fc

Framework/level distribution. CAMEL contributes 2/2/3 L1/L2/L3 cases; CrewAI 1/0/6; LangChain 5/2/1; and OpenAI Agents 4/4/6. The benchmark is intentionally not balanced: cases are admitted by historical reproducibility rather than by synthetic quota filling.

K Full RQ3 Source-Localization Results

This appendix reports the per-case source-localization artifacts used for RQ3. The same-suite attribution analysis applies the two constraints in Appendix H.1 to cases missed by the primary configuration. Because it uses the same suite, the composed results are descriptive error analysis rather than an independent benchmark estimate. The supplementary file `04_rq3_results/per_case_results.csv` contains the same records in machine-readable form.

Table 15: Completed matched RQ3 baseline runs. The supplied results contain aggregate CR@K only; per-level baseline breakdowns were not recorded in this run.

Method	CR@1	CR@3	CR@5
Raw LLM + exposed trajectory + source	5/36	10/36	12/36
HarnessFix diagnoser + HTIR + source	9/36	18/36	21/36
Proactive IHT frozen	11/36	17/36	18/36
Proactive IHT same-suite post-hoc	14/36	23/36	24/36

K.1 Cases recovered by the source-attribution refinements

The two post-hoc attribution constraints recover six base-configuration top-3 misses. The reported rank is the position of the first gold file among the ranked candidate anchors.

Table 16: Base-configuration misses recovered by the post-hoc attribution constraints.

Case	Level	Base	Post-hoc rank	Gold file(s)
camel_4013	L1	miss	1	`camel/models/xai_model.py`
crewai_3988	L1	miss	1	`lib/crewai/src/crewai/project/annotations.py`; `lib/crewai/src/crewai/project/utils.py`; `lib/crewai/src/crewai/project/wrappers.py`
crewai_4156	L3	miss	1	`lib/crewai/src/crewai/task.py`
oa_3110	L2	miss	3	`src/agents/run_internal/tool_execution.py`; `src/agents/tool.py`
oa_3961	L1	miss	2	`src/agents/function_schema.py`; `src/agents/util/_pretty_print.py`
oa_4627	L1	miss	2	`src/agents/function_schema.py`

K.2 All 36 cases

Table 17: RQ3 candidate-anchor localization in the composed same-suite artifact set by case. #Cand is the number of structured candidates; Rank is the first gold-anchor rank ("–" if no gold file appears); CR@3 marks a top-three hit.

Case	Lvl	#Cand	Rank	CR@3	Gold file(s)
camel_2605	L3	3	1	✓	camel/societies/workforce/role_playing_worker.py; camel/societies/workforce/single_agent_worker.py; camel/societies/workforce/utils.py
camel_4012	L3	4	–	–	camel/utils/langfuse.py
camel_4013	L1	3	1	✓	camel/models/xai_model.py
camel_4037	L2	3	–	–	camel/interpreters/subprocess_interpreter.py; camel/toolkits/code_execution.py
camel_4181	L1	4	–	–	camel/terminators/response_terminator.py
camel_4222	L3	3	–	–	camel/models/function_gemma_model.py
camel_4266	L2	3	–	–	camel/models/aws_bedrock_converse_model.py; camel/models/mistral_model.py
crewai_3988	L1	3	1	✓	lib/crewai/src/crewai/project/annotations.py; lib/crewai/src/crewai/project/utils.py; lib/crewai/src/crewai/project/wrappers.py
crewai_4056	L3	4	–	–	lib/crewai/src/crewai/llms/providers/azure/completion.py; lib/crewai/src/crewai/llms/providers/gemini/completion.py; lib/crewai/src/crewai/llms/providers/openai/completion.py
crewai_4156	L3	3	1	✓	lib/crewai/src/crewai/task.py
crewai_4222	L3	3	–	–	lib/crewai/src/crewai/events/event_listener.py
crewai_4389	L3	3	1	✓	lib/crewai/src/crewai/agents/crew_agent_executor.py
crewai_4788	L3	4	–	–	lib/crewai/src/crewai/llm.py
crewai_5141	L3	4	1	✓	lib/crewai/src/crewai/agents/crew_agent_executor.py
langchain_34558	L1	3	1	✓	libs/core/langchain_core/tools/base.py; libs/core/tests/unit_tests/test_tools.py
langchain_34910	L1	3	1	✓	libs/core/langchain_core/output_parsers/openai_tools.py; libs/core/tests/unit_tests/output_parsers/test_openai_tools.py
langchain_35852	L1	3	1	✓	libs/partners/anthropic/langchain_anthropic/middleware/anthropic_tools.py; libs/partners/anthropic/tests/unit_tests/middleware/test_anthropic_tools.py
langchain_35931	L1	4	3	✓	libs/core/langchain_core/tools/base.py; libs/core/langchain_core/tools/structured.py; libs/core/tests/unit_tests/test_tools.py

Case	Lvl	#Cand	Rank	CR@3	Gold file(s)
langchain_37093	L2	4	1	✓	libs/langchain_v1/langchain/agents/middleware/human_in_the_loop.py; libs/langchain_v1/tests/unit_tests/agents/middleware/implementations/test_human_in_the_loop.py
langchain_38840	L3	4	1	✓	libs/partners/perplexity/langchain_perplexity/chat_models.py; libs/partners/perplexity/tests/unit_tests/test_chat_models_responses.py
langchain_39249	L2	3	3	✓	libs/partners/anthropic/langchain_anthropic/chat_models.py; libs/partners/anthropic/tests/unit_tests/test_chat_models.py
langchain_39749	L1	3	1	✓	libs/core/langchain_core/utils/function_calling.py
oa_2384	L3	3	1	✓	src/agents/run_internal/items.py; src/agents/run_internal/turn_resolution.py
oa_2449	L1	3	–	–	src/agents/mcp/server.py
oa_2776	L3	4	3	✓	src/agents/items.py
oa_3087	L2	4	2	✓	src/agents/mcp/util.py
oa_3110	L2	3	3	✓	src/agents/run_internal/tool_execution.py; src/agents/tool.py
oa_3346	L3	3	–	–	src/agents/extensions/memory/advanced_sqlite_session.py
oa_3863	L2	3	–	–	src/agents/realtime/session.py; src/agents/run_internal/tool_execution.py; src/agents/util/_approvals.py
oa_3961	L1	4	2	✓	src/agents/function_schema.py; src/agents/util/_pretty_print.py
oa_4070	L3	3	–	–	src/agents/run.py; src/agents/run_internal/error_handlers.py; src/agents/run_internal/run_loop.py
oa_4224	L2	4	4	–	src/agents/extensions/models/any_llm_provider.py; src/agents/extensions/sandbox/blaxel/sandbox.py; src/agents/guardrail.py; src/agents/mcp/util.py; src/agents/models/chatcmpl_converter.py; src/agents/result.py; src/agents/run.py; src/agents/run_internal/guardrails.py
oa_4322	L3	4	2	✓	src/agents/run_internal/items.py; src/agents/run_internal/session_persistence.py
oa_4615	L3	3	1	✓	src/agents/memory/openai_responses_compaction_session.py; src/agents/result.py; src/agents/run.py; src/agents/run_internal/run_loop.py; src/agents/run_internal/session_persistence.py; src/agents/run_state.py
oa_4627	L1	4	2	✓	src/agents/function_schema.py

Case	Lvl	#Cand	Rank	CR@3	Gold file(s)
oa_4655	L1	3	3	✓	`src/agents/function_schema.py`

Reading the table. A rank-4 case contributes to CR@5 but not CR@3. Multi-file fixes are counted as correct if any verified changed file appears among the top-K anchors. The table exposes a useful asymmetry: several misses have plausible mechanisms but no gold file in the candidate set, which is why RQ3 is reported as file-level localization rather than semantic mechanism accuracy.

L Closed-loop Evaluation Tables

This appendix reports the detailed RQ4 results that support the compact main-paper tables: the three repeated test runs behind the matched four-backbone comparison, the Qwen3.6-Flash ablation, the full GPT-5-mini baseline table with token accounting, and the AppWorld L2/L3 evidence accounting.

L.1 Repeated runs for the matched four-backbone comparison

Each cell below records the number of successful tasks in three independent held-out test runs, followed by the mean TCR used in Table 2. The test-set sizes are 60 for GAIA, 100 for SWE-Bench Verified, 90 for AppWorld, and 34 for Terminal-Bench 2.0 Verified.

Table 18: Per-run task successes behind the matched four-backbone RQ4 comparison.

Bench.	Backbone	H_0 runs → TCR	HF runs → TCR	IHT runs → TCR
GAIA	DeepSeek-v4.1-Flash	39/41/38 → 65.6	42/47/44 → 73.9	46/44/49 → 77.2
	MiniMax-M2.5	39/42/39 → 66.7	46/43/48 → 76.1	49/47/49 → 80.6
	Qwen3.6-Flash	36/39/37 → 62.2	41/45/45 → 72.8	48/44/48 → 77.8
	HY3	42/45/43 → 72.2	49/46/48 → 79.4	50/50/48 → 82.2
SWE	DeepSeek-v4.1-Flash	63/68/67 → 66.0	72/77/73 → 74.0	78/74/78 → 76.7
	MiniMax-M2.5	65/71/68 → 68.0	78/72/76 → 75.3	79/82/76 → 79.0
	Qwen3.6-Flash	61/67/64 → 64.0	70/75/76 → 73.7	79/76/80 → 78.3
	HY3	67/72/69 → 69.3	74/80/74 → 76.0	81/76/78 → 78.3
App.	DeepSeek-v4.1-Flash	63/59/65 → 69.3	72/66/69 → 76.7	70/75/70 → 79.6
	MiniMax-M2.5	51/47/54 → 56.3	58/64/57 → 66.3	66/60/64 → 70.4
	Qwen3.6-Flash	48/51/46 → 53.7	60/55/60 → 64.8	62/67/59 → 69.6
	HY3	67/69/76 → 78.5	75/80/74 → 84.8	82/77/77 → 87.4
TB2	DeepSeek-v4.1-Flash	16/18/16 → 49.0	18/22/18 → 56.9	21/19/21 → 59.8
	MiniMax-M2.5	13/16/14 → 42.2	16/20/18 → 52.9	19/22/18 → 57.8
	Qwen3.6-Flash	16/19/17 → 51.0	22/18/21 → 59.8	20/24/21 → 63.7
	HY3	18/21/19 → 56.9	20/23/22 → 63.7	23/21/23 → 65.7

L.2 Qwen3.6-Flash ablation: repeated runs

The ablation uses Qwen3.6-Flash with `enable_thinking=true`, temperature 1.0, and `max_tokens=4096`. All variants reuse the same benchmark splits, outer repair budget, and validation gate as the matched comparison. Each entry reports the three successful-task counts followed by the mean TCR in Table 3.

Table 19: Per-run task successes for the Qwen3.6-Flash ablation.

Variant	GAIA /60	SWE /100	AppWorld /90	TB2 /34
H_0	36/39/37 → 62.2	61/67/64 → 64.0	48/51/46 → 53.7	16/19/17 → 51.0
HarnessFix	41/45/45 → 72.8	70/75/76 → 73.7	60/55/60 → 64.8	22/18/21 → 59.8
w/o multi-view evidence	45/43/44 → 73.3	75/73/75 → 74.3	61/59/61 → 67.0	21/20/21 → 60.8
L1-only routing	46/47/44 → 76.1	77/76/78 → 77.0	63/61/62 → 68.9	22/21/21 → 62.7
w/o L3 activation	48/45/46 → 77.2	79/77/78 → 78.0	65/61/61 → 69.3	22/21/22 → 63.7
w/o causal verification	46/44/45 → 75.0	77/75/77 → 76.3	62/60/61 → 67.8	20/21/22 → 61.8
Full Proactive IHT	48/44/48 → 77.8	79/76/80 → 78.3	62/67/59 → 69.6	20/24/21 → 63.7

L.3 FULL GPT-5-MINI BASELINE COMPARISON

The non-IHT rows in Table 20 are reproduced from the published HarnessFix comparison (Chen et al., 2026b); they are not rerun in this work. Proactive IHT is evaluated on the same reported benchmark splits and initial harnesses. A slash denotes that offline optimization/repair tokens are not applicable for a fixed initial or human-designed harness.

Table 20: GPT-5-mini end-to-end comparison. Published non-IHT rows are from HarnessFix; only Proactive IHT is newly evaluated here. Offline Tokens (M) excludes benchmark task-run tokens.

Benchmark	Role	Method / harness	TCR	Tokens (M)
GAIA				
	Initial	open-deep-research (H_0)	43.3	/
	Human-designed	DeepResearchAgent	52.8	/
	Human-designed	MiroFlow	58.3	/
	Evolve/repair	H_0+GEPA	46.7	32.6
	Evolve/repair	H_0+SCOPE	45.0	26.9
	Evolve/repair	H_0+ReCreate	52.8	67.9
	Evolve/repair	H_0+Meta-Harness	56.7	94.2
	Evolve/repair	H_0+HarnessFix	61.7	57.6
	Evolve/repair	H_0+Proactive IHT	**63.3**	63.7
SWE-Bench Verified				
	Initial	mini-swe-agent (H_0)	45.3	/
	Human-designed	OpenHands	47.3	/
	Human-designed	Trae-Agent	48.7	/
	Evolve/repair	H_0+GEPA	46.7	28.1
	Evolve/repair	H_0+SCOPE	48.3	25.6
	Evolve/repair	H_0+ReCreate	51.7	57.2
	Evolve/repair	H_0+Meta-Harness	54.7	82.9
	Evolve/repair	H_0+HarnessFix	57.3	48.9
	Evolve/repair	H_0+Proactive IHT	**60.3**	54.2
AppWorld				
	Initial	simplified ReAct (H_0)	36.7	/
	Human-designed	FullCodeRefl	35.6	/
	Human-designed	IPFunCall	38.1	/
	Human-designed	CUGA	41.1	/
	Evolve/repair	H_0+GEPA	37.4	22.7
	Evolve/repair	H_0+SCOPE	38.9	18.8
	Evolve/repair	H_0+ReCreate	39.3	44.1
	Evolve/repair	H_0+Meta-Harness	40.4	74.6
	Evolve/repair	H_0+HarnessFix	43.0	37.2
	Evolve/repair	H_0+Proactive IHT	**45.6**	41.9
Terminal-Bench 2.0				
	Initial	Harbor Terminus-2 (H_0)	17.6	/
	Human-designed	OpenCode	22.5	/
	Human-designed	OpenHands	18.6	/
	Evolve/repair	H_0+GEPA	19.6	18.6
	Evolve/repair	H_0+SCOPE	20.6	18.1
	Evolve/repair	H_0+ReCreate	21.6	38.8
	Evolve/repair	H_0+Meta-Harness	23.5	66.8
	Evolve/repair	H_0+HarnessFix	26.5	33.5
	Evolve/repair	H_0+Proactive IHT	**28.4**	36.3

L.4 APPWORLD ALL-TRAJECTORY AUDIT AND ACTIVATION ACCOUNTING

The AppWorld characterization uses all 90 training trajectories from the frozen DeepSeek-v4.1-Flash H_0 run. It is intentionally more exhaustive than the production repair loop: the closed-loop comparisons analyze all L1 and all L2-like trajectories but activate only two healthy/L3 candidates per optimization run. Here we analyze every L2-like route and activate every healthy/L3 candidate to estimate the diagnostic evidence available beyond failures when activation cost is not the bottleneck.

Table 21: Invariant accounting for the 22 AppWorld L2-like training trajectories. "Flags" counts auditor outputs; "Traj." counts unique routed trajectories on which the invariant fires.

Invariant	Layer	Flags	Traj.
`trace_evidence_sufficient`	Observability	22	22
`cd_codex_secret_redaction`	Governance	22	22
`no_internal_reasoning_leak`	Governance	12	12
`sandbox_boundary_kept`	Execution	8	8
`action_effect_matches_state`	Execution	8	8
`verification_not_bypassed`	Verification	4	4
`verification_result_not_ignored`	Verification	4	4
Total	–	80	22

Across layers, the 80 flags comprise 34 governance, 22 observability, 16 execution, and 8 verification outputs. Every L2-like trajectory has at least one governance and one observability flag; eight also carry execution-layer evidence and four carry verification-layer evidence. These are frozen auditor outputs, not hand-adjudicated defect labels.

Table 22: Manifestation kind by activation trigger for the 38 manifested AppWorld healthy/L3 candidates.

Trigger family	L1-like	L2-like	Total
Malformed tool call	10	10	20
Timeout	16	0	16
Instruction deviation	1	0	1
Empty response	1	0	1
Total	28	10	38

The activation records expose a strong source concentration. Thirty-five of the 38 manifestations target the observability interface, two governance, and one execution. At the source-mechanism level, 37/38 records point to functions in `appworld_agent/official_trace.py`; only one points to `appworld_agent/core.py:problem_statement`. The most frequent mechanism is `message_from_lm_output` (14 cases), followed by `read_jsonl` (6), `read_official_messages` (5), `read_official_model_calls` (4), `messages_from_model_calls` (3), `redact_lm_payload` (3), and `redact_secret_text` (2). Appendix M gives the complete candidate-level evidence chains.

M AppWorld All-Trajectory Audit and Full Activation Details

This appendix expands the AppWorld analysis in Section 7.2. The run uses the 90 training tasks of the frozen DeepSeek-v4.1-Flash H_0 configuration. Unlike the production closed-loop policy (all L1 + all L2 + two sampled L3 activations), this diagnostic-capacity study retains every L2-like trajectory and activates every healthy/L3 candidate. Its purpose is to test whether the proposed routes can expose useful non-L1 evidence and source-grounded mechanisms in a standard public workload, not to define the cost-optimal repair policy. L2-like denotes a successful trajectory with at least one active invariant flag; it is a routing decision rather than independently adjudicated defect ground truth. Likewise, an activation "manifestation" means that an executed probe produced a task-outcome change or a new relevant invariant signal under the method's manifestation contract. The supplementary material in `05_rq4_l2_l3/` contains the corresponding JSON summaries and machine-readable CSVs.

M.1 All 22 L2-like trajectories

The all-trajectory auditor routes 22/90 trajectories to L2-like evidence. Across these trajectories it emits 80 active invariant flags (mean 3.64 per trajectory). The table below preserves every routed trajectory, the invariant families that fired, and the trajectory steps on which local evidence was attached. "global" denotes a run-level check without a single message-step anchor.

Table 23: Per-trajectory AppWorld L2-like audit results. Invariant abbreviations are expanded in Table 21. The last column summarizes the number of trajectory-local evidence steps plus run-level checks; exact step indices and text are retained in the supplementary JSON/CSV.

Task ID	#	Invariant signals	Evidence anchors
09ac073_2	3	reasoning, trace, secret	8 local steps + 1 run-level
21abae1_3	4	trace, sandbox, action/state, secret	7 local steps + 1 run-level
27e1026_2	4	trace, verify-bypass, verify-result, secret	7 local steps + 3 run-level
27e1026_3	5	reasoning, trace, verify-bypass, verify-result, secret	7 local steps + 3 run-level
34d9492_3	3	reasoning, trace, secret	11 local steps + 1 run-level
3c13f5a_1	5	reasoning, trace, sandbox, action/state, secret	17 local steps + 1 run-level
4242c97_2	3	reasoning, trace, secret	8 local steps + 1 run-level
432dc7a_3	3	reasoning, trace, secret	12 local steps + 1 run-level
5800354_2	4	trace, sandbox, action/state, secret	29 local steps + 1 run-level
60d0b5b_1	4	trace, sandbox, action/state, secret	9 local steps + 1 run-level
6171bbc_3	3	reasoning, trace, secret	9 local steps + 1 run-level
80acbaf_3	4	trace, verify-bypass, verify-result, secret	25 local steps + 3 run-level
9016950_1	3	reasoning, trace, secret	15 local steps + 1 run-level
998908e_2	4	trace, sandbox, action/state, secret	15 local steps + 1 run-level
a53a8fd_1	3	reasoning, trace, secret	13 local steps + 1 run-level
d037dc8_2	4	trace, sandbox, action/state, secret	10 local steps + 1 run-level
d0b1f43_3	4	trace, sandbox, action/state, secret	13 local steps + 1 run-level
d194965_2	3	reasoning, trace, secret	13 local steps + 1 run-level
eb5ad85_3	3	reasoning, trace, secret	5 local steps + 1 run-level
f099b4c_3	3	reasoning, trace, secret	14 local steps + 1 run-level
fa327a6_2	4	trace, sandbox, action/state, secret	15 local steps + 1 run-level
fd1f8fa_2	4	trace, verify-bypass, verify-result, secret	9 local steps + 3 run-level

M.2 Full activation of all 45 healthy/L3 candidates

All 45 successful invariant-clean candidates receive the frozen activation budget. Every candidate has at least one probe applied and executed. Thirty-eight candidates produce a relevant manifestation (84.4%): 28 are L1-like outcome failures and 10 preserve task success but introduce new invariant evidence (L2-like). The seven unresolved candidates comprise four no-manifestation cases, two unsupported trigger families, and one execution/instrumentation error.

This skew is important when reading the manifestation distribution below. The activation experiment evaluates whether the method's prioritized risks can be turned into runtime evidence; it is not a random sample of harness source locations.

Table 24: Risk-candidate concentration before activation. Each healthy candidate receives eight ranked risks. The top-ranked candidate targets `official_trace.py` in all 45 cases.

Source file	All generated risks	Share	Top-1 risks
`appworld_agent/official_trace.py`	291/360	80.8%	45/45
`appworld_agent/core.py`	69/360	19.2%	0/45

Table 25: Aggregate properties of the 38 manifested AppWorld activation cases.

Trigger family	Count	Share	Target interface	Count	Share
Malformed tool call	20	52.6%	Observability	35	92.1%
Timeout	16	42.1%	Governance	2	5.3%
Instruction deviation	1	2.6%	Execution	1	2.6%
Empty response	1	2.6%			

Table 26: Source mechanisms associated with manifested activation cases. These source-grounded manifestation records are evaluated separately by the method's mechanism-verification stage.

Mechanism	Cases
`message_from_lm_output`	14
`read_jsonl`	6
`read_official_messages`	5
`read_official_model_calls`	4
`messages_from_model_calls`	3
`redact_lm_payload`	3
`redact_secret_text`	2
`problem_statement`	1

Representative manifested cases. The examples below show the evidence chain retained for qualitative inspection: the generated trigger, the observed manifestation, the source-grounded mechanism hypothesis, and its primary source anchor.

Table 27: Representative AppWorld activation cases. Full trigger specifications and source paths are in the released JSON/CSV artifacts.

Task	Probe / outcome	Mechanism / file	Evidence chain
042a9fc_3	timeout; L1_like	`read_official_messages official_trace.py`	**Trigger:** `official_logs/logger.jsonl` is being written asynchronously and read before write completes or file is partially flushed. **Observed:** `no_internal_reasoning_leak`.
0d01c76_1	malformed; L2_like	`read_jsonl official_trace.py`	**Trigger:** A log line contains valid JSON that is not a dict (e.g., a list or string), but is intended as a tool call. **Observed:** `no_internal_reasoning_leak`.
5e27cd7_3	malformed; L2_like	`message_from_lm_output official_trace.py`	**Trigger:** `choices[0]['message']` contains `tool_calls` with malformed arguments (e.g., missing `id` or invalid JSON) and `redact_.....` **Observed:** `verification_not_bypassed`, `verification_result_not_ignored`.
d0b1f43_2	malformed; L1_like	`message_from_lm_output official_trace.py`	**Trigger:** `choices[0]['message']` contains `tool_calls` with a function name not in the allowed set, e.g. `search_engine`. **Observed:** `action_effect_matches_state`, `no_internal_reasoning_leak`, `sandbox_boundary_kept`.

All candidates. Table 28 retains all 45 activation outcomes rather than only successful cases. For manifested candidates, the mechanism and source anchor come from the saved source-grounded activation record. For unresolved cases, the last targeted mechanism/source is shown only as the attempted probe target.

Table 28: Per-candidate AppWorld activation results. Mechanism is the saved source-grounded mechanism for manifested cases and the final targeted mechanism for unresolved cases; full paths are in the supplementary CSV.

Task ID	Final status	Trigger	Manifest	Mechanism	File
024c982_3	manifested	malformed	L1_like	redact_secret_text	official_trace.py
042a9fc_3	manifested	timeout	L1_like	read_official_messages	official_trace.py
09ac073_1	manifested	malformed	L2_like	messages_from_model_calls	official_trace.py
0d01c76_1	manifested	malformed	L2_like	read_jsonl	official_trace.py
0d22252_2	manifested	malformed	L1_like	message_from_lm_output	official_trace.py
1150ed6_1	no-manif.	empty	–	read_official_model_calls	official_trace.py
1b988ac_2	manifested	timeout	L1_like	message_from_lm_output	official_trace.py
22cc237_1	manifested	timeout	L1_like	message_from_lm_output	official_trace.py
23d431c_3	manifested	malformed	L1_like	read_jsonl	official_trace.py
258796c_3	no-manif.	malformed	–	read_official_messages	official_trace.py
2ff4dfb_2	manifested	malformed	L2_like	message_from_lm_output	official_trace.py
325d6ec_1	manifested	malformed	L1_like	message_from_lm_output	official_trace.py
3650990_3	manifested	malformed	L1_like	redact_lm_payload	official_trace.py
3b8fb7a_1	manifested	timeout	L1_like	message_from_lm_output	official_trace.py
3d9a636_3	manifested	instr.dev.	L1_like	problem_statement	core.py
4d12842_1	no-manif.	malformed	–	redact_secret_text	official_trace.py
5e27cd7_3	manifested	malformed	L2_like	message_from_lm_output	official_trace.py
634f342_2	manifested	timeout	L1_like	read_jsonl	official_trace.py
7434096_3	manifested	timeout	L1_like	message_from_lm_output	official_trace.py
7574325_1	unsupported	stale	–	redact_lm_payload	official_trace.py
9126bf0_3	no-manif.	halluc.	–	message_from_lm_output	official_trace.py
986aa4e_1	manifested	malformed	L2_like	read_official_model_calls	official_trace.py
986aa4e_3	manifested	timeout	L1_like	message_from_lm_output	official_trace.py
9bf2c8a_2	manifested	timeout	L1_like	read_official_model_calls	official_trace.py
a53a8fd_2	manifested	timeout	L1_like	read_official_messages	official_trace.py
a8f302f_2	manifested	timeout	L1_like	read_official_messages	official_trace.py
aaa25aa_1	manifested	malformed	L1_like	redact_lm_payload	official_trace.py
af84964_1	manifested	malformed	L2_like	read_jsonl	official_trace.py
b4707e8_3	manifested	empty	L1_like	message_from_lm_output	official_trace.py
b6d1104_2	manifested	timeout	L1_like	message_from_lm_output	official_trace.py

Task ID	Final status	Trigger	Manifest	Mechanism	File
b9c5c9a_3	exec-error	wrong-type	–	redact_lm_payload	official_trace.py
cb4ee7a_1	manifested	malformed	L1_like	read_official_messages	official_trace.py
ccf4b82_2	manifested	malformed	L2_like	messages_from_model_calls	official_trace.py
cef9191_3	manifested	malformed	L2_like	read_official_model_calls	official_trace.py
cf6abd2_3	manifested	timeout	L1_like	read_official_messages	official_trace.py
d0b1f43_2	manifested	malformed	L1_like	message_from_lm_output	official_trace.py
d6d8cb6_3	manifested	timeout	L1_like	messages_from_model_calls	official_trace.py
d8e490b_2	manifested	malformed	L1_like	read_official_model_calls	official_trace.py
d8e490b_3	manifested	timeout	L1_like	message_from_lm_output	official_trace.py
dbc0276_3	manifested	malformed	L2_like	read_jsonl	official_trace.py
dcb2559_1	unsupported	permission	–	resolve_appworld_root	core.py
dcbd34b_1	manifested	timeout	L1_like	redact_lm_payload	official_trace.py
ec437da_1	manifested	timeout	L1_like	redact_secret_text	official_trace.py
efc3cea_2	manifested	malformed	L1_like	message_from_lm_output	official_trace.py
f323bae_2	manifested	malformed	L2_like	read_jsonl	official_trace.py

Unresolved cases. Four candidates complete executable probes without producing a relevant manifestation (1150ed6_1, 258796c_3, 4d12842_1, and 9126bf0_3). Two candidates use actuator modes outside the supplied AppWorld execution interface (7574325_1: stale-state; dcb2559_1: permission-boundary). One candidate (b9c5c9a_3) ends in an execution/instrumentation error. These outcomes are retained as activation records rather than treated as negative defect labels.